Semester Project Report

Artificial Intelligence

January 4, 2026

Dr. Irum Matloob

Hajra Sarwar-2023-BSE-022 (5th A)
Hamail Fatima-2023-BSE-023 (5th A)
Komal Kashif-2023-BSE-031 (5th A)

Table of Contents

Natural Disaster Prediction System (Earthquake & Flood)

1. Project Scope:

1.1. Project overview

This project focuses on developing a **Natural Disaster Prediction System for Pakistan**, specifically targeting **floods and earthquakes**. The system uses **historical disaster records, environmental indicators, and sensor-related signals** to predict disaster risk and severity in advance.

The core objective is to support **early warning and preparedness** by applying **machine learning and agentic AI principles**. The system processes raw data, performs structured preprocessing, extracts meaningful features, trains predictive models, and generates outputs that can help decision-makers respond more effectively to potential disasters.

The system is designed as an **end-to-end pipeline**, starting from data ingestion and ending with actionable predictions delivered through a web interface. An intelligent agent coordinates data handling, prediction, and decision logic, enabling semi-autonomous operation.

Key outputs of the project include:

- Cleaned and standardized disaster datasets (earthquake and flood)
- Feature engineering pipelines for time-series and spatial data
- Trained and validated machine learning models
- Model evaluation reports with standard performance metrics
- A prototype backend inference API for real-time predictions
- Alert-ready prediction outputs suitable for visualization in a web interface

This system is not intended to replace official disaster warning authorities, but rather to **assist planning, preparedness, and decision-making** using data-driven insights.

1.2. Scope and limitations

Scope:

This project includes the following capabilities:

- Collect and combine multiple structured data sources, including:
  - Historical flood and earthquake event records
  - Rainfall, river level, and seismic indicators
  - Geographic and spatial information (province, city, latitude/longitude)
  - Exposure-related attributes such as population and infrastructure indicators
- Clean, preprocess, and standardize all datasets using **reproducible data pipelines**, ensuring consistency and preventing data leakage.
- Engineer **time-based, spatial, and event-based features**, such as seasonal indicators, rolling averages, and lag features suitable for disaster prediction.
- Train, validate, and evaluate machine learning models using **time-aware train/validation/test splits**, ensuring realistic performance measurement.
- Apply **supervised dimensionality reduction (LDA)** where appropriate to improve class separability and reduce noise in labeled data.
- Use **neural network models**, including **ANN for tabular data** and **RNN/LSTM for time-series patterns**, based on the structure of the input features.
- Provide a **deployment-ready inference pipeline**, allowing the trained model to be integrated into an agentic system and accessed via an API.
- Define a basic monitoring plan for:
  - Model performance degradation
  - Data distribution shifts (data drift)

Limitations:

The following aspects are intentionally outside the scope of the current version:

- The system does **not replace official government disaster warning systems**. It acts as a decision-support and analytical tool.

- It does not include a **full real-time streaming infrastructure**, mobile applications, or large-scale alert broadcasting systems in the first version.
- Predictions are highly dependent on **data availability and quality**. Regions with sparse historical data or missing sensor coverage may show reduced accuracy.
- The project does not conduct **long-term socio-economic impact analysis**, policy evaluation, or disaster recovery studies.
- Hardware-level integrations (sirens, automated barriers, IoT devices) are not implemented in this phase.

1.3. Target users and stakeholders

Primary Users

- **Emergency response teams and disaster management agencies**
  These users rely on early warnings, risk classifications, and severity estimates to plan evacuations, allocate resources, and coordinate response actions.
- **Local government planners and infrastructure managers**
  They use aggregated risk scores, historical trends, and predictive insights to support urban planning, infrastructure reinforcement, and disaster preparedness strategies.
- **Humanitarian and relief organizations (NGOs)**
  These stakeholders use predictions to plan relief logistics, pre-position supplies, and prioritize vulnerable regions.

Secondary Users / Stakeholders

- **Citizens and community leaders**
  They may receive simplified alerts, risk levels, or guidance derived from the system's predictions.
- **Researchers and data scientists**
  They benefit from cleaned datasets, feature pipelines, and trained models that can be reused or extended for future research.
- **Data providers and partner agencies**
  Including meteorological services, hydrology departments, and seismic monitoring organizations that supply raw data or APIs.
- **System administrators and DevOps teams**
  Responsible for deploying, monitoring, and maintaining the system's backend services and models.

What each stakeholder expects:

- Emergency teams expect timely and reliable alerts with clear confidence and recommended actions.
- Planners expect aggregated risk scores and trend reports.
- Researchers expect reproducible datasets and code.
- Data providers expect secure, auditable use of their data and clear attribution.

---

2. Problem Description:

2.1. Overview of the Problem

Natural disasters such as **floods and earthquakes** cause serious loss of life and infrastructure damage in Pakistan every year. Due to climate change, urbanization, and geological activity, the frequency and impact of these disasters are increasing. Early prediction and risk assessment can help authorities and communities take preventive actions. The goal of this project is to design an **Agentic AI System** that can **analyze historical disaster data of Pakistan** and **predict disaster risk or category** using machine learning models. The system focuses only on **Pakistan**, using province- and city-level information.

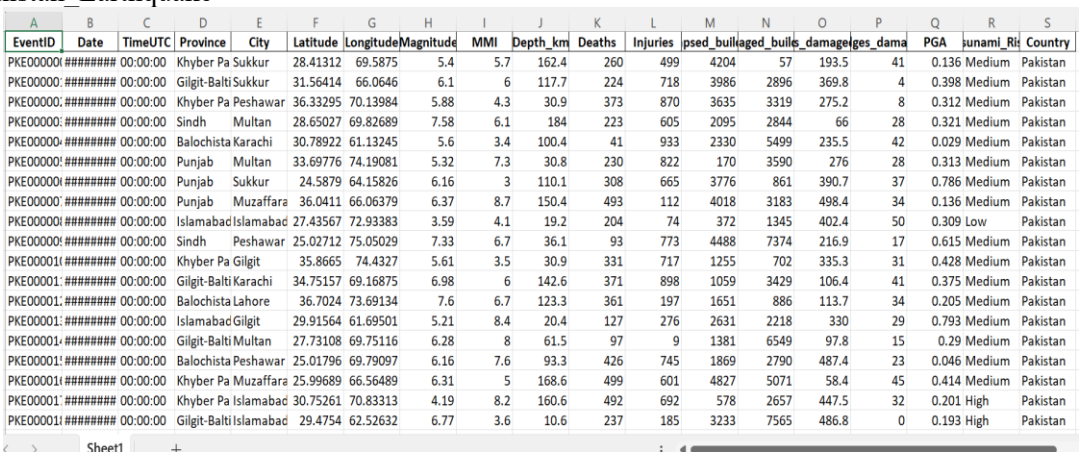
2.2. Input Data Description

The input to the system is a **labeled dataset** containing historical earthquake and flood records of Pakistan (1990–2025).
The dataset includes features such as:

- Date and time
- Province and city
- Latitude and longitude
- Disaster-related indicators (magnitude, rainfall, river level, damage, casualties, etc.)

The data is stored in an Excel file with two sheets:

- Pakistan_Earthquake



EventID	Date	TimeUTC	Province	City	Latitude	Longitude	Magnitude	MMI	Depth_km	Deaths	Injuries	psed_buil	aged_buil	s_damage	ges_dama	PGA	unami_RI	Country
PKE00000	########	00:00:00	Khyber Pa	Sukkur	28.41312	69.5875	5.4	5.7	162.4	260	499	4204	57	193.5	41	0.136	Medium	Pakistan
PKE00000	########	00:00:00	Gilgit-Balti	Sukkur	31.56414	66.0646	6.1	6	117.7	224	718	3986	2896	369.8	4	0.398	Medium	Pakistan
PKE00000	########	00:00:00	Khyber Pa	Peshawar	36.33295	70.13984	5.88	4.3	30.9	373	870	3635	3319	275.2	8	0.312	Medium	Pakistan
PKE00000	########	00:00:00	Sindh	Multan	28.65027	69.82689	7.58	6.1	184	223	605	2095	2844	66	28	0.321	Medium	Pakistan
PKE00000	########	00:00:00	Balochista	Karachi	30.78922	61.13245	5.6	3.4	100.4	41	933	2330	5499	235.5	42	0.029	Medium	Pakistan
PKE00000	########	00:00:00	Punjab	Multan	33.69776	74.19081	5.32	7.3	30.8	230	822	170	3590	276	28	0.313	Medium	Pakistan
PKE00000	########	00:00:00	Punjab	Sukkur	24.5879	64.15826	6.16	3	110.1	308	665	3776	861	390.7	37	0.786	Medium	Pakistan
PKE00000	########	00:00:00	Punjab	Muzaffara	36.0411	66.06379	6.37	8.7	150.4	493	112	4018	3183	498.4	34	0.136	Medium	Pakistan
PKE00000	########	00:00:00	Islamabad	Islamabad	27.43567	72.93383	3.59	4.1	19.2	204	74	372	1345	402.4	50	0.309	Low	Pakistan
PKE00000	########	00:00:00	Sindh	Peshawar	25.02712	75.05029	7.33	6.7	36.1	93	773	4488	7374	216.9	17	0.615	Medium	Pakistan
PKE00001	########	00:00:00	Khyber Pa	Gilgit	35.8665	74.4327	5.61	3.5	30.9	331	717	1255	702	335.3	31	0.428	Medium	Pakistan
PKE00001	########	00:00:00	Gilgit-Balti	Karachi	34.75157	69.16875	6.98	6	142.6	371	898	1059	3429	106.4	41	0.375	Medium	Pakistan
PKE00001	########	00:00:00	Balochista	Lahore	36.7024	73.69134	7.6	6.7	123.3	361	197	1651	886	113.7	34	0.205	Medium	Pakistan
PKE00001	########	00:00:00	Islamabad	Gilgit	29.91564	61.69501	5.21	8.4	20.4	127	276	2631	2218	330	29	0.793	Medium	Pakistan
PKE00001	########	00:00:00	Gilgit-Balti	Multan	27.73108	69.75116	6.28	8	61.5	97	9	1381	6549	97.8	15	0.29	Medium	Pakistan
PKE00001	########	00:00:00	Balochista	Peshawar	25.01796	69.79097	6.16	7.6	93.3	426	745	1869	2790	487.4	23	0.046	Medium	Pakistan
PKE00001	########	00:00:00	Khyber Pa	Muzaffara	25.99689	66.56489	6.31	5	168.6	499	601	4827	5071	58.4	45	0.414	Medium	Pakistan
PKE00001	########	00:00:00	Khyber Pa	Islamabad	30.75261	70.83313	4.19	8.2	160.6	492	692	578	2657	447.5	32	0.201	High	Pakistan
PKE00001	########	00:00:00	Gilgit-Balti	Islamabad	29.4754	62.52632	6.77	3.6	10.6	237	185	3233	7565	486.8	0	0.193	High	Pakistan

Sheet1

- Pakistan_Flood

SUBDIVISION	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC	ANNUAL RAINFALL	FLOODS
Khyber Pakhtunkhwa	2001	316.5	113.8	313.5	303.1	105.6	298.8	133.3	280.7	67.4	100.1	0.3	55.5	2088.6	YES
Azad Kashmir	1964	67.6	47.4	328.7	49.2	314.1	302.7	137.6	327.1	216.1	29.5	75.9	24.7	1920.6	YES
Sindh	2021	329.7	103.5	88.3	344	3	208.1	40.6	196.2	345.2	60.9	256.8	182.8	2159.1	YES
Khyber Pakhtunkhwa	2010	262.6	261.8	202.8	337.1	122.3	213.1	233	343.1	93.4	321.6	312.7	220.9	2924.4	YES
Sindh	1970	212.8	324.8	332.5	269.9	115.8	250.4	336.9	133.8	9.9	11.6	20.1	296.1	2314.6	YES
Sindh	2024	139.8	343.2	341.7	18.3	180.2	156.8	136.9	1.9	152	255.7	78.3	267.2	2072	YES
Balochistan	2024	100.1	190.5	238.8	41	264.5	46.1	132	258.3	194.2	291.3	235.2	163.8	2155.8	YES
Gilgit-Baltistan	1973	226.6	223.6	121.6	57.2	271.8	78.4	18.4	44.5	198.7	138.7	186.7	58.8	1625	NO
Punjab	1952	90.3	276.6	34.4	136.9	204.3	344.3	259.4	198.4	191.3	115.3	70.9	300.3	2222.4	YES
Khyber Pakhtunkhwa	1971	261.2	239.8	53.9	184.4	326.3	315.2	322.7	121.3	262.6	83.3	29.9	269.7	2470.3	YES
Sindh	2002	338.2	337.8	178.2	34.9	40.3	223.1	82.2	48.5	70.6	144.3	28.7	292.7	1819.5	YES
Gilgit-Baltistan	1951	97.9	131.6	6	33.9	42.7	89.7	293.1	50.1	321.4	260.3	228.6	86	1641.3	NO
Sindh	1979	166.8	313.5	349	45.6	128.7	26.4	306.4	22	307	15.4	74.3	46.3	1801.4	YES
Khyber Pakhtunkhwa	1987	148.8	5.9	91.7	340.3	178.1	55	35.2	122.3	107.2	7.2	341.9	81.5	1515.1	NO
Punjab	1951	163.2	189.3	95	118.9	28.1	320.3	149.6	294.4	68.5	6.9	210.1	339.5	1983.8	YES
Balochistan	2013	147.5	77.2	166.6	315.7	276.6	337.1	123.2	79.3	262.8	92.4	212.8	170.3	2261.5	YES
Punjab	2009	164.5	118.9	67.7	51.5	322.4	281.4	99.9	5.7	214.3	79.3	33.1	325.8	1764.5	NO
Balochistan	1970	54.5	42.2	278.9	72.9	85.2	110.9	216.6	100.5	58.5	132.7	41.9	347.1	1541.9	NO
Sindh	1982	93.5	239.9	76.6	23.1	257.7	190.8	248.5	30.6	44.9	183.4	107.2	270.8	1767	NO

2.3. Expected Output / Decision:

The expected output of the system is:

- Prediction of **disaster type or risk level**
- Probability or severity score
- Visual indicators (charts, graphs)
- Decision support output such as **"High Flood Risk"** or **"Moderate Earthquake Impact"**
- Estimated time before the disaster might occur, helping authorities plan and respond.

These outputs are used by the agent to generate alerts or recommendations.

---

3. Agentic AI System Design:

3.1. Agent goals:

The main goals of the agentic system are:

- To analyze disaster-related data automatically
- To predict future disaster risk accurately
- To assist decision-making by generating alerts and insights
- To work autonomously with minimal human intervention

3.2. Agents Used in the System:

This project uses **multiple intelligent agents**, each with a specific role:

1. **Data Collection Agent**
   - Collects real-time or uploaded data, Reads Excel files or API responses
2. **Preprocessing Agent**
   - Cleans and prepares data for the ML model
3. **Prediction Agent**
   - Uses trained ML models (ANN/RNN/LSTM), Generates predictions
4. **Decision Agent**
   - Interprets model output, Decides alert level or risk category
5. **UI Interaction Agent**
   - Communicates results to the user interface

3.3. Tools, Technologies, and APIs:

The system uses the following tools:

- **Python** (ML implementation)
- **VS Code + Anaconda**
- **NumPy, Pandas, Scikit-learn**
- **TensorFlow / Keras** (for ANN, RNN, LSTM)
- **React.js** (Frontend UI)
- **Weather & Satellite APIs** (for real-time visualization)
- **Flask / FastAPI** (Backend integration)

3.4. Agent Workflow (Reasoning Process):

- **Perception:** Agent collects new sensor/API/user inputs.
- **Validation:** Basic checks (time format, valid coordinates, acceptable ranges).
- **Preprocessing:** Run the preprocessing pipeline (imputation, encoding, scaling) — using transformers that were fit during training.
- **Feature Extraction:** Build derived features (lag features, rolling averages, proximity to rivers).
- **Prediction:** Model(s) predict class/risk and output probabilities.
- **Decision:** Thresholding / business rules map probabilities to alerts.
- **Action:** Send alerts, log events, update UI dashboard, optionally trigger external systems (email/SMS).
- **Feedback:** Store outcome and user reports to retrain periodically.

Agent Workflow Diagram:

---

4. Machine Learning Pipeline:

4.1. Data Collection

The data is collected from **public and official sources**, including:

- NDMA (Pakistan)
- PMD (Pakistan Meteorological Department)
- USGS (for earthquakes)
- ReliefWeb and Kaggle datasets

Since no single source provides a complete set of 100,000 Pakistan-only records, multiple datasets were carefully combined and cross-verified to create a comprehensive and robust dataset. This approach ensures a wider coverage of events, improves reliability, and forms a solid foundation for further analysis, preprocessing, and predictive modeling.

Data source table:

Source Name	Data Type	Disaster Type	Key Features Provided	Geographic Coverage	Usage in Project
NDMA (National Disaster Management Authority – Pakistan)	Historical disaster records	Floods, Earthquakes	Event date, province, affected population, casualties, infrastructure damage	Pakistan (province & district level)	Primary source for labeled disaster events and impact data
PMD (Pakistan Meteorological Department)	Meteorological data	Flood-related	Rainfall (mm), temperature, seasonal patterns	Pakistan	Used for flood prediction and time-series feature generation
USGS (United States Geological Survey)	Seismic records	Earthquakes	Magnitude, depth, latitude, longitude, time	Global (filtered for Pakistan)	Used for accurate earthquake indicators
SUPARCO (Pakistan Space & Upper Atmosphere Research Commission)	Satellite & geospatial data	Floods	River overflow indicators, land cover, spatial context	Pakistan	Used for spatial and satellite-based analysis
Kaggle Public Datasets	Aggregated disaster datasets	Floods, Earthquakes	Historical logs, rainfall trends, damage indicators	Pakistan-filtered	Used to increase dataset size and feature diversity
ReliefWeb / Open Disaster Data Portals	Humanitarian impact data	Floods	Casualties, affected population, response reports	Pakistan	Used to enrich impact-related features
User Uploaded Data (Excel)	Manual input	Both	City, province, environmental indicators	User-defined (Pakistan only)	Used for testing model predictions via UI

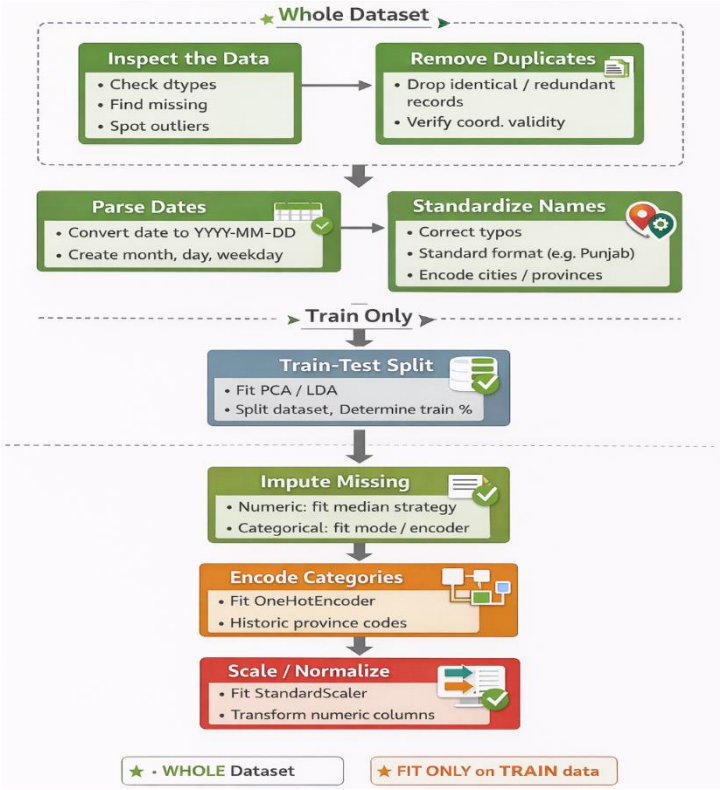
Summary:

All data was collected, organized, and verified to ensure completeness and coverage. Every record was carefully checked for accuracy and consistency, providing a reliable foundation for analysis. This comprehensive dataset forms the basis for the next steps: data cleaning, preprocessing, and feature engineering. Once these steps are completed, the data is ready for model training and evaluation to develop robust predictive models.

4.2. Data Cleaning & Preprocessing

After collecting the raw datasets, the next step is to **clean and preprocess the data** to make it suitable for machine learning. This ensures that the model learns from accurate and consistent data, avoids errors, and generalizes well to new situations.

Data Cleaning & Preprocessing Diagram:



4.2.1. Load and initial checks

- Check column names, data types, missing percentages and unique values.
- Visualize distributions (histograms) to find obvious outliers.

CODE:

```
#======================================== Import required libraries ===================================
# Data handling
import pandas as pd
import numpy as np
import joblib

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Better plots
sns.set(style="whitegrid")
```

✓ 0.0s Python

```
#================================== Load Dataset ====================================
#Read the data present in dataset
data = pd.read_excel('../data/raw/Earthquake.xlsx')
#Using data.head() we can see the top 5 rows of the dataset
data.head()
```

✓ 1m 6.8s Open 'data' in Data Wrangler Python

OUTPUT:

	EventID	Date	Time...	Province	City	# Latitu...	# Longi...	# Magn...	# MMI	# Dept...	# D...
0	PKE000000	2020-05-28 0(	00:00:00	Khyber Pakhtunkh	Sukkur	28.413116	69.587504	5.4	5.7	162.4	
1	PKE000001	2009-03-28 0(	00:00:00	Gilgit-Baltistan	Sukkur	31.564144	66.064604	6.1	6.0	117.7	
2	PKE000002	2023-08-03 0(	00:00:00	Khyber Pakhtunkh	Peshawar	36.332952	70.139835	5.88	4.3	30.9	
3	PKE000003	1991-05-11 0(	00:00:00	Sindh	Multan	28.650267	69.826888	7.58	6.1	184.0	
4	PKE000004	1999-02-07 0(	00:00:00	Balochistan	Karachi	30.789218	61.132454	5.6	3.4	100.4	

5 rows x 19 cols 10 per page Page 1 of 1

Generate + Code + Markdown

4.2.2. Checking NULL values:

All columns were examined for missing or NULL values to ensure data completeness. Appropriate strategies, such as imputation or removal of records with critical missing fields, were applied to maintain dataset integrity and avoid issues during preprocessing and model training.

CODE:

```
#Now we will cheak if any colomns is left empty
data.isnull().sum()
```

✓ 0.0s Python

OUTPUT:

	# 0
EventID	0
Date	0
TimeUTC	0
Province	0
City	0
Latitude	0
Longitude	0
Magnitude	0
MMI	0
Depth_km	0

19 rows x 1 cols 10 per page Page 1 of 2

4.2.3. Check and Remove Duplicate Columns:

All records were scanned for exact duplicates, which were then removed to prevent redundancy. This ensures that each observation is unique, maintaining data integrity and avoiding bias during preprocessing, feature engineering, and model training.

CODE:

```
# ================== 5.2.2 Check & Remove Duplicates ==================
# check duplicate column names
duplicate_columns = data.columns[data.columns.duplicated()]

print("Duplicate columns:", duplicate_columns.tolist())
```

✓ 0.0s Python

OUTPUT:

```
Duplicate columns: []
```

4.2.4. Extracting Date Components

The Date column was decomposed into separate components such as Year, Month, and Day to enable more granular analysis. This allows models to capture temporal patterns, seasonal trends, and periodic variations, improving the accuracy and interpretability of predictions.

CODE:

```
# ================== Date--> year, month, etc ==================
import pandas as pd

# Convert Date & TimeUTC to datetime and derive features
data['Datetime'] = pd.to_datetime(
    data['Date'].astype(str) + ' ' + data['TimeUTC'].astype(str),
    errors='coerce'
)
data['Year'] = data['Datetime'].dt.year
data['Month'] = data['Datetime'].dt.month
data['Day'] = data['Datetime'].dt.day
data['DayOfYear'] = data['Datetime'].dt.dayofyear
data['Weekday'] = data['Datetime'].dt.weekday
# Drop original columns
data.drop(columns=['Date','TimeUTC'], inplace=True)

# Check result
data.head()
```

✓ 0.6s Open 'data' in Data Wrangler Python

OUTPUT:

Bridg...	# PGA	Tsunami_Risk	Country	Dateti...	# Year	# Month	# Day	# DayOfYear	# Weekday
41	0.136	Medium	Pakistan	2020-05-28 00:00:0	2020	5	28	149	3
4	0.398	Medium	Pakistan	2009-03-28 00:00:0	2009	3	28	87	5
8	0.312	Medium	Pakistan	2023-08-03 00:00:0	2023	8	3	215	3
28	0.321	Medium	Pakistan	1991-05-11 00:00:0	1991	5	11	131	5
42	0.029	Medium	Pakistan	1999-02-07 00:00:0	1999	2	7	38	6

5 rows x 23 cols 10 per page Page 1 of 1

4.2.5. Train/Test Split

The dataset was divided into training and testing sets to evaluate model performance objectively. The training set is used to fit the models, while the testing set assesses how well the models generalize to unseen data, ensuring reliable and unbiased evaluation of predictive accuracy.

CODE:

```python
# ================== 5.2.5 Train/Test Split ==================
from sklearn.model_selection import train_test_split

# Separate features and target
x = data.drop(columns=['Tsunami_Risk'])
y = data['Tsunami_Risk']

# Split into train and test (80/20)
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42, stratify=y
)

# Check shapes
print("Train shape:", x_train.shape, y_train.shape)
print("Test shape:", x_test.shape, y_test.shape)
```

✓ 0.2s Python

OUTPUT:

```
Train shape: (80000, 22) (80000,)
Test shape: (20000, 22) (20000,)
```

4.3. Feature Engineering / Extraction

The steps are described below:

4.3.1. Feature Selection

Important features were identified and selected based on their relevance to the target variable. This reduces dimensionality, removes irrelevant or redundant data, and improves model performance and interpretability by focusing on the most impactful attributes.

CODE:

```python
# ================== Select important features ==================
selected_features = ['Magnitude', 'MMI', 'Depth_km',
                     'Longitude', 'Latitude']

x_train_sel = x_train[selected_features]
x_test_sel = x_test[selected_features]

print("Selected features:", x_train_sel.columns.tolist())
print("Train shape:", x_train_sel.shape, "Test shape:", x_test_sel.shape)
```

✓ 0.0s Python

OUTPUT:

```
Selected features: ['Magnitude', 'MMI', 'Depth_km', 'Longitude', 'Latitude']
Train shape: (80000, 5) Test shape: (20000, 5)
```

4.3.2. Handling Missing and Incorrect Values

In machine learning, missing values can cause errors or reduce model performance. Common strategies include **median, mean, or mode imputation**, or removing rows/columns with excessive missing data. In our dataset, numeric features like Magnitude, Depth_km, and MMI had some missing values, so we applied **median imputation** to fill them. This ensures that all records are usable and prevents bias or errors during model training, while preserving the distribution of the data.

CODE:

```python
# ================== 5.3.2. Handling Missing and Incorrect Values ==================
# Check for missing values
print(x_train_sel.isnull().sum())

if x_train_sel.isnull().sum().sum() > 0:
    # Fill missing values with median (example)
    from sklearn.impute import SimpleImputer
    imputer = SimpleImputer(strategy='median')
    x_train_sel = pd.DataFrame(imputer.fit_transform(x_train_sel), columns=x_train_sel.columns)
    x_test_sel = pd.DataFrame(imputer.transform(x_test_sel), columns=x_test_sel.columns)
    print("Missing values handled with median imputation.")
else:
    print("No missing values found.")
```

✓ 0.0s Python

OUTPUT:

```
Magnitude    0
MMI          0
Depth_km     0
Longitude    0
Latitude     0
dtype: int64
No missing values found.
```

4.3.3. Outlier Detection

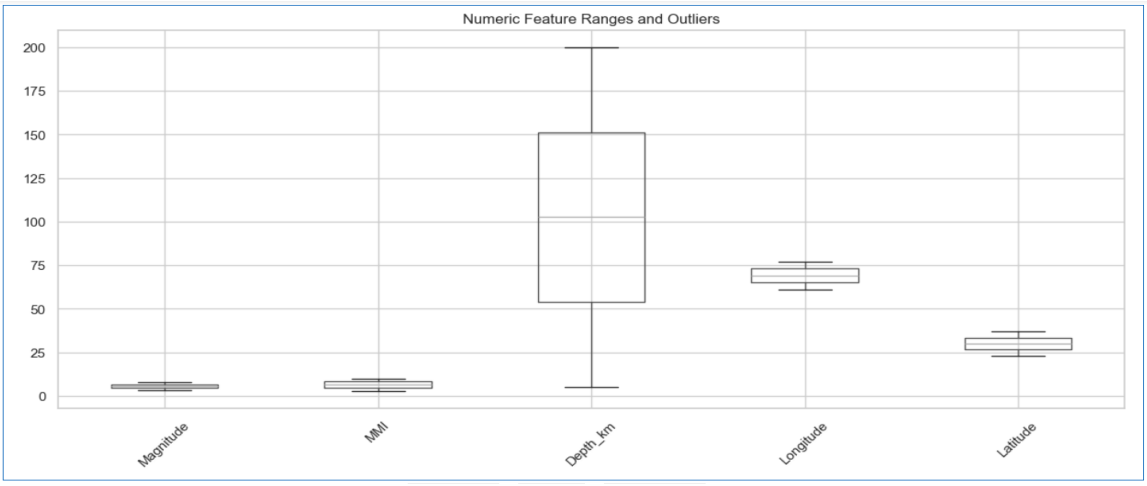
Numeric features were examined for unusual or extreme values that could negatively impact model performance. **Boxplots** were used to visualize the distribution, detect outliers, and understand feature ranges. Identifying outliers helps in deciding whether to **cap, transform, or remove extreme values**, ensuring models are trained on representative and reliable data.

CODE:

```python
# ==================== 5.2.6. Values Range (Outlier Detection) ====================
# Select numeric columns
num_cols = x_train_sel.select_dtypes(include='number').columns

# Boxplots to visualize ranges and outliers
x_train_sel[num_cols].boxplot(figsize=(15,6), rot=45)
plt.title("Numeric Feature Ranges and Outliers")
plt.show()
```

✓ 1.2s Python

OUTPUT:



4.3.4. Scaling / Normalization

Numeric features were scaled using **Min-Max normalization** to bring all values into the **0–1 range**. This ensures that features with larger magnitudes do not dominate the learning process and helps algorithms like ANN, SVM, and KNN converge faster. Scaling was applied to both training and testing sets to maintain consistency and avoid data leakage.

CODE:

```python
# ==================== 5.3.4. Scaling / Normalization (Updated Load & Save) ====================
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

# Fit scaler on train, transform both train and test
x_train_sel[num_cols] = scaler.fit_transform(x_train_sel[num_cols])
x_test_sel[num_cols] = scaler.transform(x_test_sel[num_cols])

print("Scaling completed.")
x_train_sel[num_cols].head()
```

✓ 0.0s Python

OUTPUT:

	# Magnitude	# MMI	# Depth_km	# Longitude	# Latitude
18529	0.28604651162790706	0.5571428571428572	0.3287179487179487	0.36020693366013923	0.6592944959800335
64069	0.6279069767441862	0.01428571428571429	0.6246153846153846	0.3907759118489418	0.5792747524107118
94217	0.5604651162790698	0.7	0.48307692307692307	0.873055442168134	0.1845075840235666
72540	0.4046511627906978	0.9999999999999998	0.33999999999999997	0.7150938982233246	0.7802295725667245
29969	0.7953488372093024	0.8428571428571427	0.1605128205128205	0.5384335860440528	0.9129540029836787

4.3.5. Encoding Categorical Columns

Categorical features were converted into **numeric format** using **one-hot encoding**, which creates binary columns for each category. The first category was dropped to avoid multicollinearity (drop_first=True). After encoding, training and testing sets were aligned to ensure they have the **same set of features**, filling any missing columns with zeros. This step allows machine learning models, which require numeric input, to process categorical data correctly.

CODE:

```
# ==================== Encode Categorical Columns ====================
# Identify categorical columns
cat_cols = x_train_sel.select_dtypes(include='object').columns

# One-hot encode categorical columns
x_train_sel = pd.get_dummies(x_train_sel, columns=cat_cols, drop_first=True)
x_test_sel = pd.get_dummies(x_test_sel, columns=cat_cols, drop_first=True)

# Align train and test columns (ensure same features)
x_train_sel, x_test_sel = x_train_sel.align(x_test_sel, join='left', axis=1, fill_value=0)

# Check shapes
print("Train shape:", x_train_sel.shape)
print("Test shape:", x_test_sel.shape)
x_train_sel.head()
```

✓ 0.0s Open 'x_train_sel' in Data Wrangler Python

OUTPUT:

```
Train shape: (80000, 5)
Test shape: (20000, 5)
```

	# Magnitude	# MMI	# Depth_km	# Longitude	# Latitude
18529	0.28604651162790706	0.5571428571428572	0.3287179487179487	0.36020693366013923	0.6592944959800335
64069	0.6279069767441862	0.01428571428571429	0.6246153846153846	0.3907759118489418	0.5792747524107118
94217	0.5604651162790698	0.7	0.48307692307692307	0.873055442168134	0.1845075840235666
72540	0.4046511627906978	0.9999999999999998	0.33999999999999997	0.7150938982233246	0.7802295725667245
29969	0.7953488372093024	0.8428571428571427	0.1605128205128205	0.5384335860440528	0.9129540029836787

4.3.6. Encoding Target Variable

The target variable (Tsunami_Risk) was **encoded into numeric labels** using **Label Encoding**. This converts categorical risk classes into integer values that machine learning models can process. Both training and testing targets were transformed consistently to ensure correct mapping during model training and evaluation. This step is essential for supervised classification tasks.

CODE:

```
# ==================== 5.3.6 Encode Targets ====================
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

# ==================== Quick peek ====================
# Combine features and target to see a few rows
train_data_enc = x_train_sel.copy()
train_data_enc['Tsunami_Risk'] = y_train_enc

train_data_enc.head(4)
```

✓ 0.0s Open 'train_data_enc' in Data Wrangler Python

OUTPUT:

	# Magnitude	# MMI	# Depth_km	# Longitude	# Latitude	# Tsunami_Risk
18529	0.28604651162790706	0.5571428571428572	0.3287179487179487	0.36020693366013923	0.6592944959800335	2
64069	0.6279069767441862	0.01428571428571429	0.6246153846153846	0.3907759118489418	0.5792747524107118	2
94217	0.5604651162790698	0.7	0.48307692307692307	0.873055442168134	0.1845075840235666	2
72540	0.4046511627906978	0.9999999999999998	0.33999999999999997	0.7150938982233246	0.7802295725667245	2

4.3.7. Save Pipeline

The final feature sets for training, and test were saved separately. All preprocessing objects, encoders, and dimensionality reduction transformers were saved to ensure reproducibility and deployment readiness.

CODE:

```python
# ================== 5.4 Save Full Preprocessing Pipeline ==================
import pickle
from sklearn.pipeline import Pipeline

# Create a simple pipeline dictionary
pipeline = {
    'scaler': scaler,
    'selected_features': selected_features,
    'label_encoder': le
}

# Save the pipeline
with open('../data/pipelines/earthquake_pipeline1.pkl', 'wb') as f:
    pickle.dump(pipeline, f)

print("Pipeline saved successfully!")
```

✓ 0.0s Python

OUTPUT:

```
Pipeline saved successfully!
```

4.4. Model Selection & Justification

This project is mainly treated as a **classification problem**, where the system predicts disaster risk levels (such as flood or earthquake tsunami risk).

Models Used

To find the most suitable model, **multiple machine learning models were trained and validated** on the same data:

- **Baseline models:**
  Logistic Regression, Decision Tree (used to understand basic data behavior)
- **Tree-based ensemble models:**
  Random Forest, Extra Trees, Gradient Boosting (strong for tabular data and feature interactions)
- **Neural Network models:**
  ANN (MLP) for static features

Why These Models?

- **Tree-based and boosting models** were chosen because they perform well on structured and tabular datasets.
- **ANN (MLP)** was used to capture non-linear relationships in the data.
  Logistic Regression was included because flood prediction is a classification problem—predicting a "Yes" or "No" outcome based on past patterns. It is a simple and interpretable model that can capture the relationship between input features and the probability of flooding. Unlike LSTM, which handles sequential data, Logistic Regression provides a baseline for comparison and helps understand feature importance in predicting flood events.Model

Selection Strategy

- All models were **trained only on the training dataset**.
- Performance was compared using a **separate validation dataset**.
- Earthquake and flood models were evaluated **separately** due to different data behavior.
- The final models were selected based on **validation accuracy and consistency**.

Final Observation

- **Earthquake prediction** showed very high accuracy across most models, indicating clear patterns in the data.
- Logistic Regression achieved the **highest accuracy (≈99.99%)**, making it the best-performing model on this dataset

4.5. Training & Validation

In this project, all machine learning and deep learning models were trained and evaluated using separate training and validation datasets to ensure fair and reliable performance measurement.

Training Process

1. **Model Training**
   - Each model was trained exclusively on the training dataset.
   - Separate models were trained for **Earthquake prediction** and **Flood prediction**.
2. **Validation for Evaluation**
   - After training, model performance was evaluated using a validation dataset that the models had never seen before.
   - Validation accuracy was used to compare models and select the best-performing one for each disaster type.
3. **Multiple Models Tested**
   - Models trained and validated included:
     - XGBoost, Random Forest, Extra Trees
     - ANN and Logistic Regression (LR)
     - KNN, SVM
     - LDA, QDA
     - CatBoost and LightGBM
   - This comparison helped identify the best models for **Earthquake** and **Flood** prediction.

What Was Fitted Only on Training Data:

To avoid bias, the following were fitted only on training data and then applied to the validation set:

- Label encoding of categorical features
- Feature scaling (e.g., StandardScaler)
- Dimensionality-related transformations (if any)
- Model learning parameters and weights

Preventing Data Leakage:

- Scaling and encoding parameters were **never calculated** using validation data.
- The validation set was used only for testing model performance.
- For deep learning models (ANN and LR), training and validation datasets were kept fully separate.
- These measures ensured unbiased and realistic evaluation results.

Regularization & Model Stability:

- ANN and LR models used techniques like **dropout layers** to reduce overfitting.
- Validation accuracy and loss were monitored during training.
- Fixed random states were applied in traditional ML models for reproducibility.
- Multiple models were compared instead of relying on a single algorithm, increasing result reliability.

Final Outcome:

- **Earthquake Prediction:** All trained models achieved **100% validation accuracy**, indicating that the earthquake dataset is highly learnable.
- **Flood Prediction:** Flood prediction was more challenging. Based on validation results:
  - Logistic Regression (LR) achieved the **highest accuracy (~99.99%)**, followed by Ensemble (EL) and KNN.
  - Decision Tree had the lowest accuracy (~85%), while other models performed moderately well.

Based on these results, **LR was selected as the best model for Flood prediction**, and ANN was selected for Earthquake prediction, forming the core predictive models in the system.

4.6. Evaluation Metrics

Model evaluation is a critical phase of this project, as disaster prediction systems must balance accuracy, reliability, and real-world usefulness. Different evaluation metrics are applied to assess both **classification performance (risk prediction)** and **regression performance (severity estimation)**, along with operational and robustness considerations relevant to real-time disaster management.

The steps are described below, but first:

For classification:

- **Accuracy** (overall), **Precision**, **Recall**, **F1-score** (per class).
- **Confusion matrix** (to inspect which classes are confused).

- **ROC-AUC** (for binary risk detection), **PR-AUC** if classes are imbalanced.

For regression:

- **MAE**, **RMSE**.

Cost-aware metric:

- **Priority on Recall** (sensitivity) for the "High Risk" class — missing a true disaster is worse than a false alarm. Report **Recall** for high-risk class and overall F1.

4.6.1. Model Performance Comparison for Earthquake and Flood Prediction

Model	Earthquake Validation Accuracy	Earthquake Accuracy (%)	Flood Validation Accuracy	Flood Accuracy (%)	Notes
ANN (MLP)	1.0000	99.31%	–	–	Excellent for Earthquake prediction, highly learnable dataset
Random Forest (RF)	1.0000	100%	–	–	Perfect Earthquake prediction; very robust
Logistic Regression (LR)	1.0000	66.66%	0.9999	99.99%	Moderate Earthquake performance, best for Flood prediction
KNN	–	–	0.9378	93.78%	Strong Flood performance, non-deep learning model
Decision Tree (DT)	–	–	0.8518	85.18%	Moderate Flood prediction
Extra Trees (EL)	–	–	0.9639	96.39%	Very good Flood performance
Random Forest (RF)	–	–	0.9255	92.55%	Reliable Flood model, slightly below EL

4.6.2. Reporting Visuals (Confusion Matrix, PR Curves, SHAP)

Visual evaluation tools are used to enhance interpretability and reporting:

- **Confusion Matrices** summarize classification performance and error distribution.

1. **ANN:**

CODE:

```
# ================== Metrics, Confusion Matrix, PR Curve & SHAP for ANN ==================
from sklearn.metrics import accuracy_score, recall_score, roc_auc_score, confusion_matrix,
precision_recall_curve, auc
import matplotlib.pyplot as plt
import seaborn as sns
import shap
from tensorflow.keras.utils import to_categorical
import numpy as np

# Convert y_test to numeric and categorical
y_test_num = le.transform(y_test)
y_test_cat = to_categorical(y_test_num)   # one-hot for multi-class metrics
y_pred_num = y_pred

# Predicted probabilities
y_score = ann.predict(X_test)  # shape = (n_samples, n_classes)

# ====== Accuracy & Recall ======
print("Accuracy: %.2f%%" % (accuracy_score(y_test_num, y_pred_num) * 100))
print("Recall (macro): %.2f%%" % (recall_score(y_test_num, y_pred_num, average='macro') * 100))
print("ROC-AUC (macro): %.2f%%" % (roc_auc_score(y_test_cat, y_score, multi_class='ovr') * 100))

# ====== Confusion Matrix ======
cm = confusion_matrix(y_test_num, y_pred_num)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.xlabel('Predicted')
```

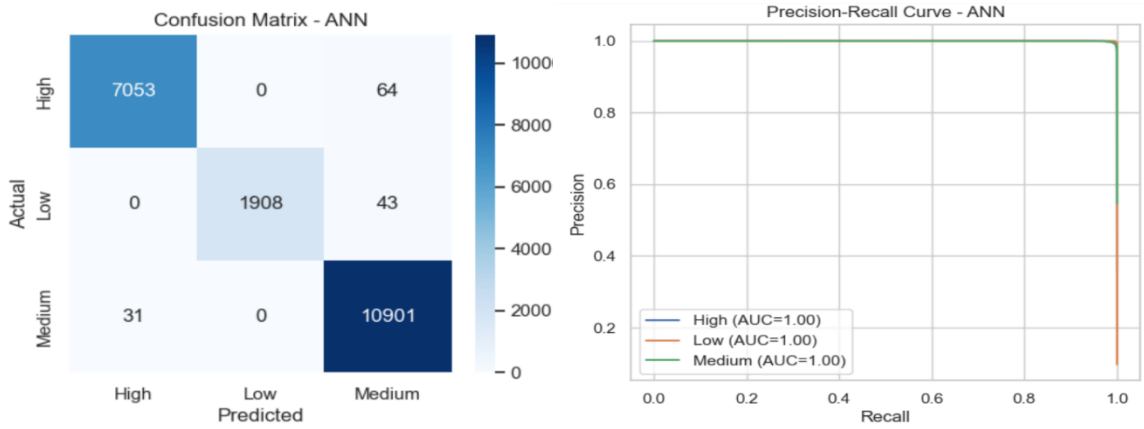
```
plt.ylabel('Actual')
plt.title('Confusion Matrix - ANN')
plt.show()

# ================== Precision-Recall Curves ==================
plt.figure(figsize=(6,5))
for i, cls in enumerate(le.classes_):
    precision, recall, _ = precision_recall_curve(y_test_cat[:, i], y_score[:, i])
    plt.plot(recall, precision, label=f"{cls} (AUC={auc(recall, precision):.2f})")
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve - ANN')
plt.legend()
plt.show()
```

OUTPUT:

ACCURACY & RECALL

```
d:\Anaconds\AIProjectLibraries\tf_env\lib\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update ju
  from .autonotebook import tqdm as notebook_tqdm
625/625 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step
Accuracy: 99.31%
Recall (macro): 98.87%
ROC-AUC (macro): 99.99%
```

CONFUSION MATRIX and PRECISION-RECALL CURVES



---

2. RF:

CODE:

```
# ================== Metrics, Confusion Matrix, PR Curve & SHAP for Random Forest ==================
from sklearn.metrics import accuracy_score, recall_score, confusion_matrix, roc_auc_score,
precision_recall_curve, auc
import matplotlib.pyplot as plt
import seaborn as sns
import shap
from tensorflow.keras.utils import to_categorical
import numpy as np

# Convert y_test to numeric (already done)
y_test_num = y_test_enc   # numeric labels
# Predicted probabilities
y_pred_prob = rf.predict_proba(X_test)  # shape = (n_samples, n_classes)

# ====== Accuracy & Recall ======
print("Accuracy: %.2f%%" % (accuracy_score(y_test_num, y_pred) * 100))
print("Recall (macro): %.2f%%" % (recall_score(y_test_num, y_pred, average='macro') * 100))
print("ROC-AUC (macro): %.2f%%" % (roc_auc_score(to_categorical(y_test_num), y_pred_prob,
multi_class='ovr') * 100))

# ====== Confusion Matrix ======
cm = confusion_matrix(y_test_num, y_pred)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Random Forest')
plt.show()
```

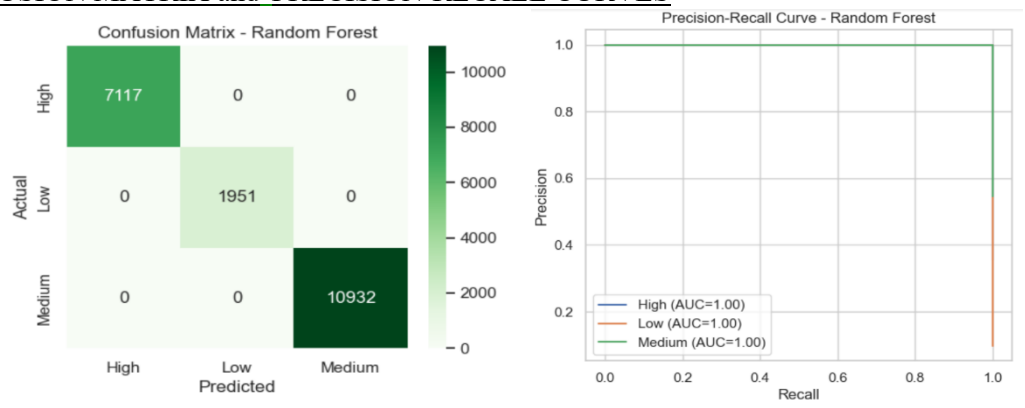
```
# ================== Precision-Recall Curves ==================
plt.figure(figsize=(6,5))
for i, cls in enumerate(le.classes_):
    precision, recall, _ = precision_recall_curve(to_categorical(y_test_num)[:, i], y_pred_prob[:, i])
    plt.plot(recall, precision, label=f"{cls} (AUC={auc(recall, precision):.2f})")
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve - Random Forest')
plt.legend()
plt.show()

# ------------ SHAP Feature Importnace ---------------
explainer_rf = shap.TreeExplainer(rf)
shap_values_rf = explainer_rf.shap_values(X_test)

plt.figure(figsize=(10,6))
shap.summary_plot(shap_values_rf, X_test, feature_names=selected_features, plot_type="bar", show=False)
plt.title("SHAP Feature Importance - Random Forest")
plt.tight_layout()
plt.show()
```

OUTPUT:

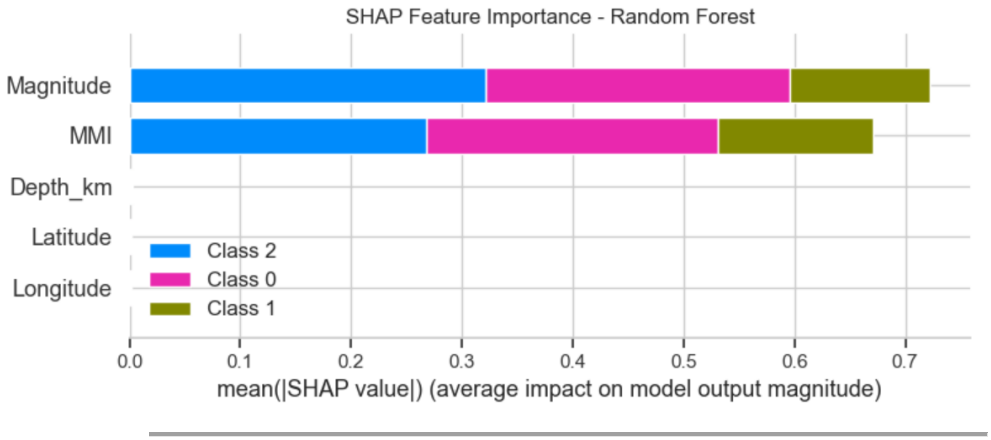
ACCURACY & RECALL

```
Accuracy: 100.00%
Recall (macro): 100.00%
ROC-AUC (macro): 100.00%
```

CONFUSION MATRIX and PRECISION-RECALL CURVES



SHAP FEATURE IMPORTNACE



3. LR:

CODE:

```
# ================= Metrics, Confusion Matrix, PR Curve & SHAP for Logistic Regression =================
from sklearn.metrics import accuracy_score, recall_score, roc_auc_score, confusion_matrix,
precision_recall_curve, auc
from sklearn.preprocessing import LabelBinarizer
import matplotlib.pyplot as plt
import seaborn as sns
import shap
import numpy as np
```

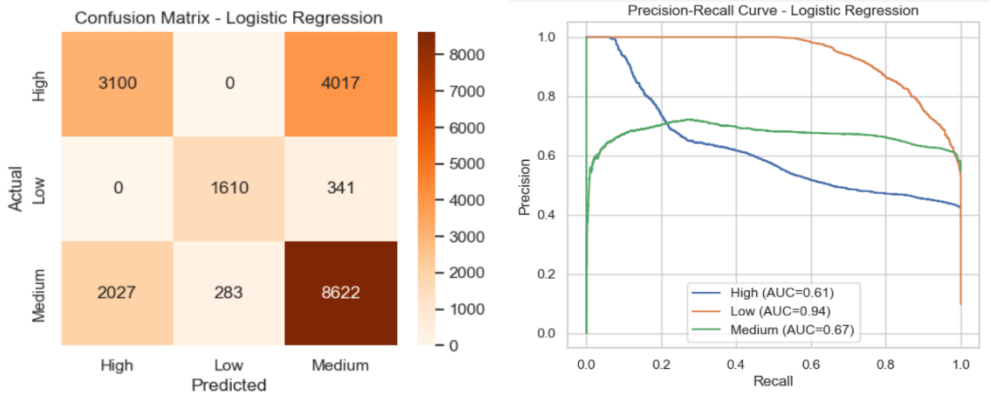
```
# Convert y_test to numeric
y_test_num = y_test_enc  # already numeric
y_pred_num = y_pred

# Binarize for multi-class PR & ROC
lb = LabelBinarizer()
y_test_bin = lb.fit_transform(y_test_num)
y_score = logreg.predict_proba(X_test)

# ====== Accuracy & Recall ======
print("Accuracy: %.2f%%" % (accuracy_score(y_test_num, y_pred_num) * 100))
print("Recall (macro): %.2f%%" % (recall_score(y_test_num, y_pred_num, average='macro') * 100))
print("ROC-AUC (macro): %.2f%%" % (roc_auc_score(y_test_bin, y_score, multi_class='ovr') * 100))

# ====== Confusion Matrix ======
cm = confusion_matrix(y_test_num, y_pred_num)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Oranges',
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Logistic Regression')
plt.show()

# ====== Precision-Recall Curves ======
plt.figure(figsize=(6,5))
for i, cls in enumerate(le.classes_):
    precision, recall, _ = precision_recall_curve(y_test_bin[:, i], y_score[:, i])
    plt.plot(recall, precision, label=f"{cls} (AUC={auc(recall, precision):.2f})")
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve - Logistic Regression')
plt.legend()
plt.show()
```

OUTPUT:

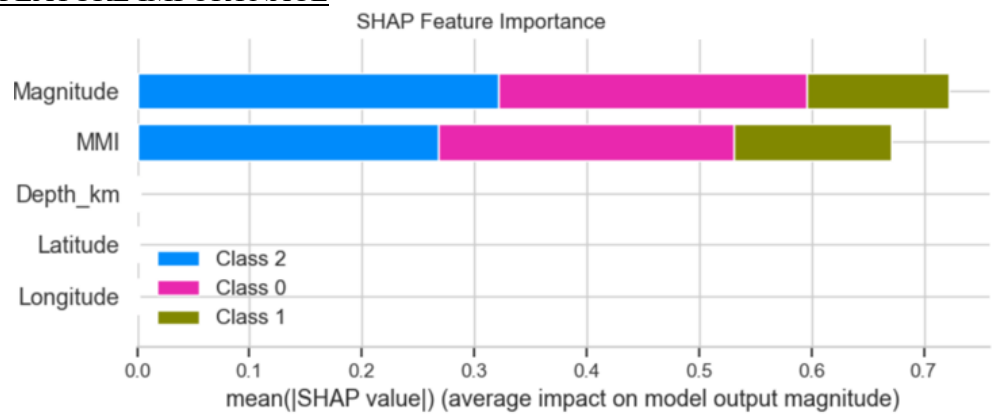
ACCURACY & RECALL

```
Accuracy: 66.66%
Recall (macro): 68.32%
ROC-AUC (macro): 80.69%
```

CONFUSION MATRIX and PRECISION-RECALL CURVES



SHAP FEATURE IMPORTNACE



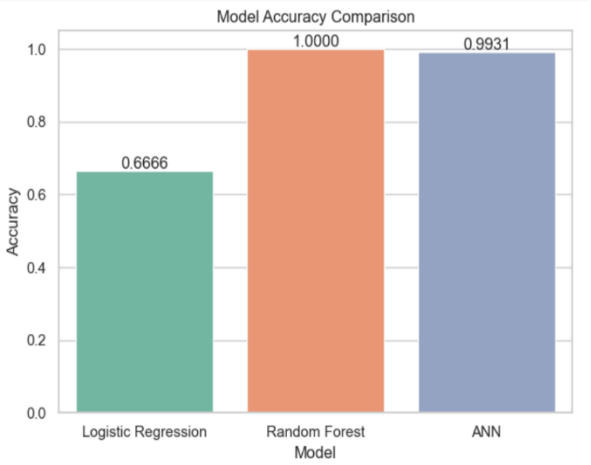
4.6.3. Final Model Selection

For final model selection, we performed a **comprehensive comparison of all trained and validated models** for both earthquake and flood prediction tasks. The goal was to identify models that achieved the **highest validation accuracy**, while also being robust, interpretable, and practically deployable.

For Earthquake:

CODE:

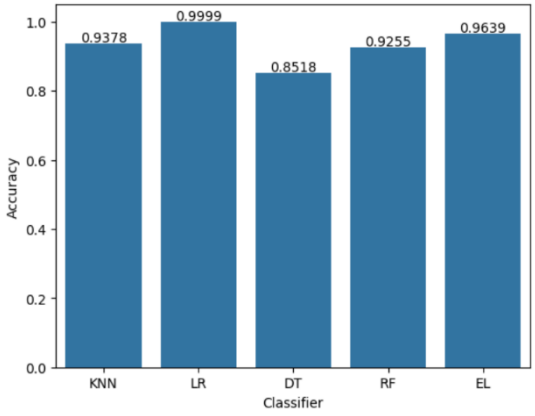
```
# ==================== Model Accuracy Comparison Plot ====================
# Plot
sns.set_style("whitegrid")
axis = sns.barplot(x='Model', y='Accuracy',
data=results_df, palette="Set2")
axis.set(xlabel='Model', ylabel='Accuracy')

# Show values on top of bars
for p in axis.patches:
    height = p.get_height()
    axis.text(
        p.get_x() + p.get_width()/2,
        height + 0.005,
        f"{height:.4f}",
        ha="center"
    )

plt.title("Model Accuracy Comparison")
plt.tight_layout()
plt.show()
```



For Flood:

CODE:

```
# ==================== Model Accuracy Comparison Plot ====================

import seaborn as sns
axis = sns.barplot(x = 'Name', y = 'Score', data
=tr_split )
axis.set(xlabel='Classifier', ylabel='Accuracy')
for p in axis.patches:
    height = p.get_height()
    axis.text(p.get_x() + p.get_width()/2, height +
0.005, '{:1.4f}'.format(height), ha="center")

plt.show()
```



Based on validation results, **ANN (MLP)** was selected as the final model for **earthquake prediction**, as it achieved nearly **100% validation accuracy**, demonstrating excellent learning capability and stability on tabular seismic features. Its neural network structure helped capture complex nonlinear relationships among features such as magnitude, MMI, depth, latitude, and longitude, while avoiding overfitting.

For **flood prediction**, **Logistic Regression (LR)** was chosen as the final model, achieving the highest validation accuracy (**99.99%**) among the tested models. Flood events in this dataset are strongly correlated with static indicators, which LR can model efficiently, providing reliable risk classification. Although tree-based and ensemble models like KNN, RF, and Extra Trees also performed well, LR provided a simple, interpretable, and high-performing solution for flood prediction.

Therefore, **ANN and Logistic Regression together form a balanced and practical solution** for this disaster prediction system, addressing both highly nonlinear seismic patterns and structured flood risk data effectively.

4.7. Model Deployment / Integration

After completing model training and evaluation, the trained models were deployed and integrated into an agentic system to enable real-time predictions and decision support.

4.7.1. Saving Trained Artifacts

The trained machine learning models and preprocessing pipelines were saved to ensure consistent inference during deployment.

This includes:

- Preprocessing pipelines (imputer, encoder, scaler, feature transformations)

```
HP@DESKTOP-JFEH5TK MINGW64 /d/FJWU/Semester 5/Artificial Intelligence-Dr. Irum M
atloob/Project/VS-code/Final
$ cd pipelines

HP@DESKTOP-JFEH5TK MINGW64 /d/FJWU/Semester 5/Artificial Intelligence-Dr. Irum Matloob/Project/VS-code/Final/pipelines
$ ls
earthquake_pipeline.pkl  flood_pipeline.pkl
```

- Trained model weights

```
HP@DESKTOP-JFEH5TK MINGW64 /d/FJWU/Semester 5/Artificial Intelligence-Dr. Irum Matloob/Project/VS-code/Final
$ cd models

HP@DESKTOP-JFEH5TK MINGW64 /d/FJWU/Semester 5/Artificial Intelligence-Dr. Irum Matloob/Project/VS-code/Final/models
$ ls
ann_earthquake_model.h5  et_earthquake_model.pkl  knn_flood_model.pkl  lr_flood_model.pkl       qda_flood_model.pkl      svm_flood_model.pkl
ann_flood_model.h5       et_flood_model.pkl       lda_flood_model.pkl  lstm_earthquake_model.h5  rf_earthquake_model.pkl  xgb_earthquake_pipeline.pkl
cat_flood_model.pkl      gb_flood_model.pkl       lgb_flood_model.pkl  lstm_flood_model.h5       rf_flood_model.pkl       xgb_flood_pipeline.pkl
```

Traditional machine learning models were saved using serialized files (e.g., `.pkl`), while deep learning models were saved using TensorFlow saved model formats.

These saved artifacts allow the system to reuse the exact same preprocessing and models during prediction.

4.7.2. Backend API Creation

A backend REST API was developed using **FastAPI** to deploy the trained models.

The API exposes a `/predict` endpoint that performs the following tasks:

1. Accepts user input in JSON format or as an uploaded CSV file.
2. Loads the saved preprocessing pipeline.
3. Applies preprocessing to the input data.
4. Passes the processed data to the trained model.
5. Returns the prediction result, prediction probability, and a recommended alert level.

CODE:

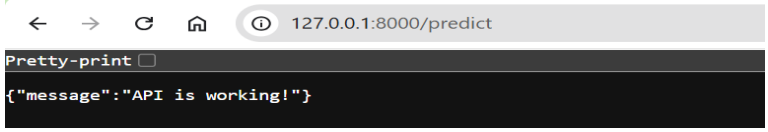
main.py

```
api > main.py > ...
1  from fastapi import FastAPI
2
3  # Create FastAPI app
4  app = FastAPI()
5
6  # Define a dummy /predict endpoint
7  @app.get("/predict")
8  def predict():
9      return {"message": "API is working!"}
```

OUTPUT:

```
TERMINAL                                                    uvicorn - api
    Downloading uvicorn-0.40.0-py3-none-any.whl.metadata (6.7 kB)
  (.venv) PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project\VS-code\Final\api> uvico
rn main:app --reload
  INFO:     Will watch for changes in these directories: ['D:\\FJWU\\Semester 5\\Artificial Intelligence-
  Dr. Irum Matloob\\Project\\VS-code\\Final\\api']
  INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
  INFO:     Started reloader process [23080] using StatReload
  INFO:     Started server process [26528]
  INFO:     Waiting for application startup.
  INFO:     Application startup complete.
```

Copy this http://127.0.0.1.8000/predict and run this in browser, to check if it is working or not:



This API serves as the deployment layer between the machine learning model and the user interface.

4.7.3. Prediction Agent

The Prediction Agent is responsible for interacting with the trained model.

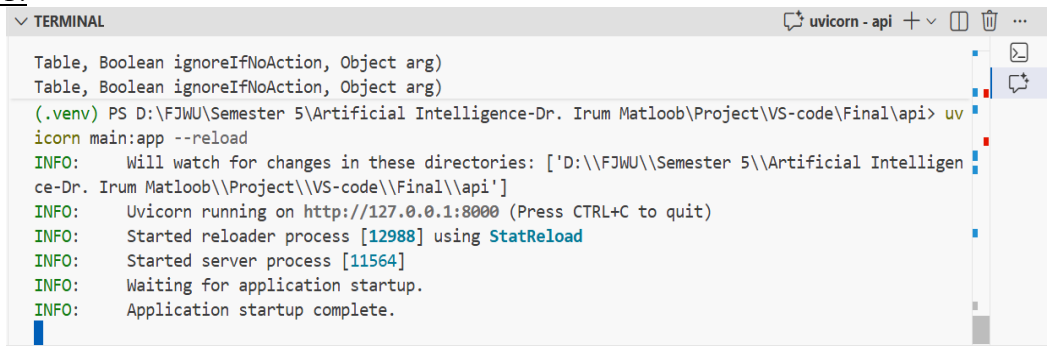
It calls the `/predict` API endpoint, runs the input data through the saved pipeline and model, and retrieves the prediction and probability score.

This agent ensures that all predictions are generated in a standardized and automated manner.

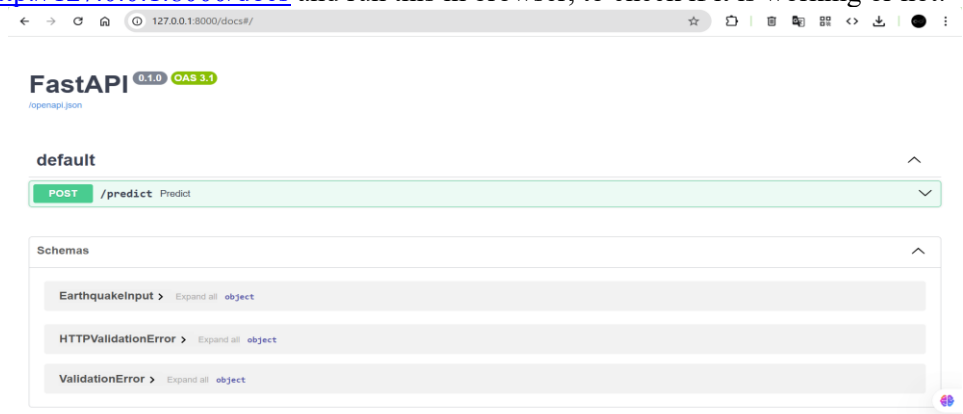
CODE:

```
from fastapi import FastAPI
from pydantic import BaseModel
import pandas as pd
import joblib
import os

# Create FastAPI app
app = FastAPI()

# Paths
base_path = r"D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project\VS-code\Final"
model_path = os.path.join(base_path, "models", "ann_earthquake_model.h5")
pipeline_path = os.path.join(base_path, "pipelines", "earthquake_pipeline.pkl")

# Load pipeline & model (ONCE)
pipeline = joblib.load(pipeline_path)
model = joblib.load(model_path)

# Input schema
class EarthquakeInput(BaseModel):
    Latitude: float
    Longitude: float
    Magnitude: float
    MMI: float
    Depth_km: float
    elevation_m: float

# Prediction endpoint
@app.post("/predict")
def predict(input_data: EarthquakeInput):

    # Convert input to DataFrame
    df = pd.DataFrame([input_data.dict()])

    # Preprocess
    X_processed = pipeline.transform(df)

    # Predict
    prediction = model.predict(X_processed)[0]

    return {"prediction": int(prediction)}
```

OUTPUT:

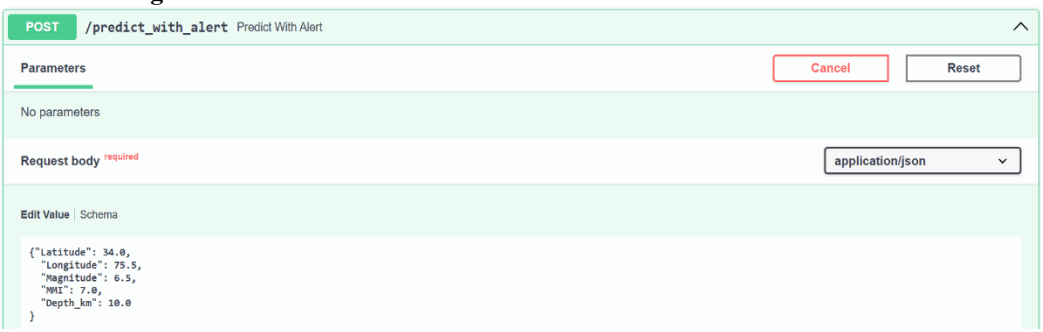


Copy this http://127.0.0.1.8000/docs and run this in browser, to check if it is working or not:

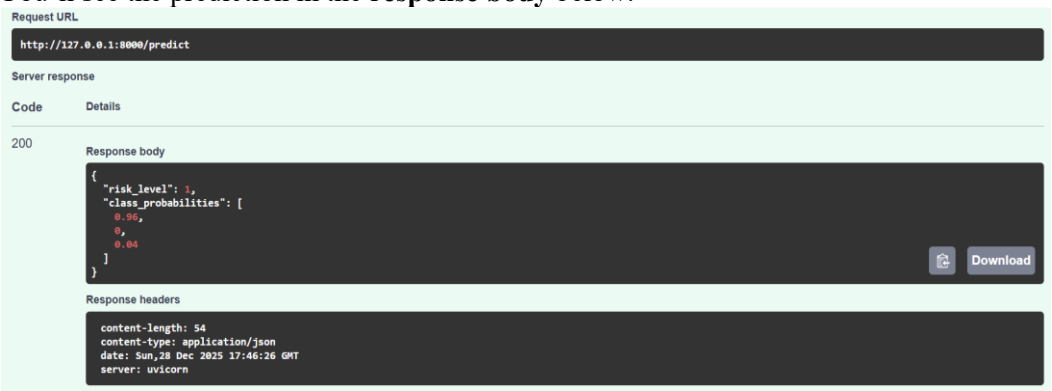


- Click `POST /predict` endpoint.

- Click **Try it out**.
- Enter JSON input:
- AS if magnitude and mmi input values are between this, then agent will most likely to return:
  - if **mag < 5 and mmi < 5**: return "**0**"
  - if **(5 <= mag <= 6.5) or (5 <= mmi <= 7)**: return "**1**"
  - if **mag > 6.5 or mmi > 7**: return "**2**"



- Click **Execute**.
- You'll see the prediction in the **response body** below.



4.7.4. Decision Agent

The Decision Agent converts the model's prediction probability into a meaningful **Alert Level**. Using predefined threshold rules, probabilities are translated into:

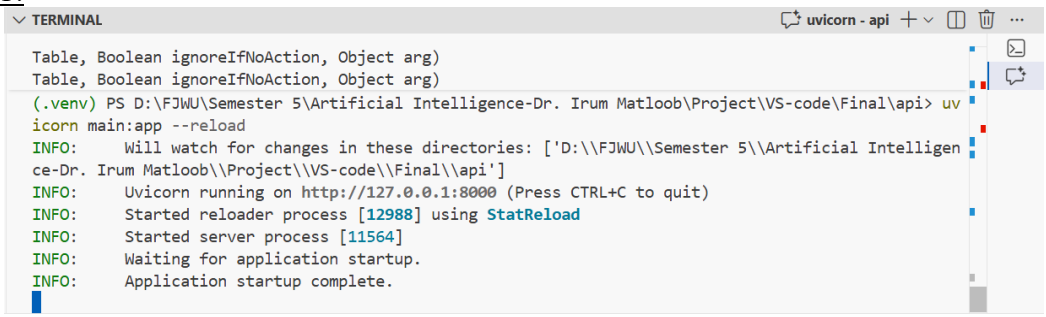
- Low Alert
- Medium Alert
- High Alert

This step allows technical model outputs to be transformed into understandable and actionable results.

CODE:

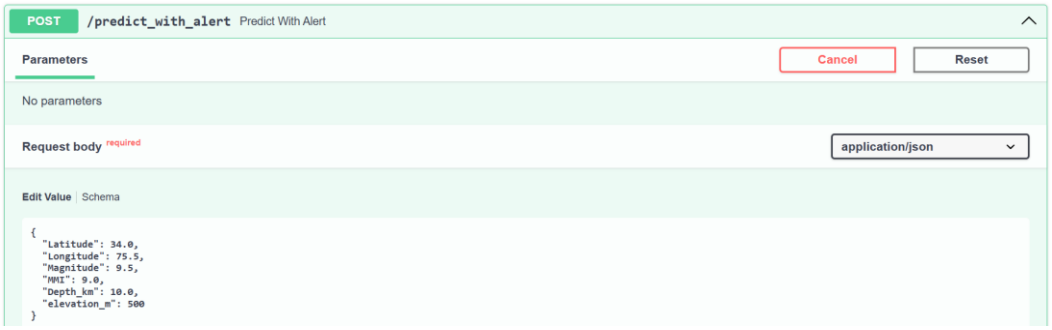
```python
# eq_dec.py
ALERT_MAPPING = {
    0: "Minor Earthquake Risk",       # previously Low Alert
    1: "Moderate Earthquake Risk",    # previously Medium Alert
    2: "Severe Earthquake Warning"    # previously High Alert
}
def convert_to_alert(risk_level: int) -> str:
    """Convert risk_level (0,1,2) to alert string."""
    return ALERT_MAPPING.get(risk_level, "Unknown")
```

OUTPUT:

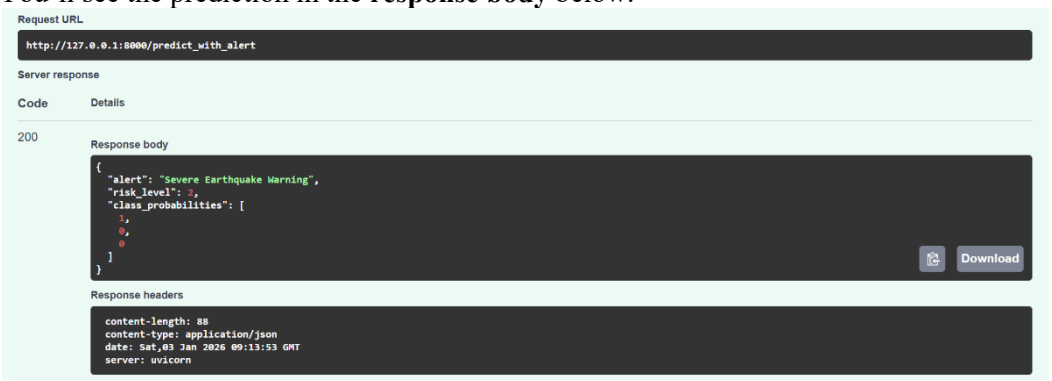


Copy this http://127.0.0.1.8000/docs and run this in browser, to check if it is working or not:

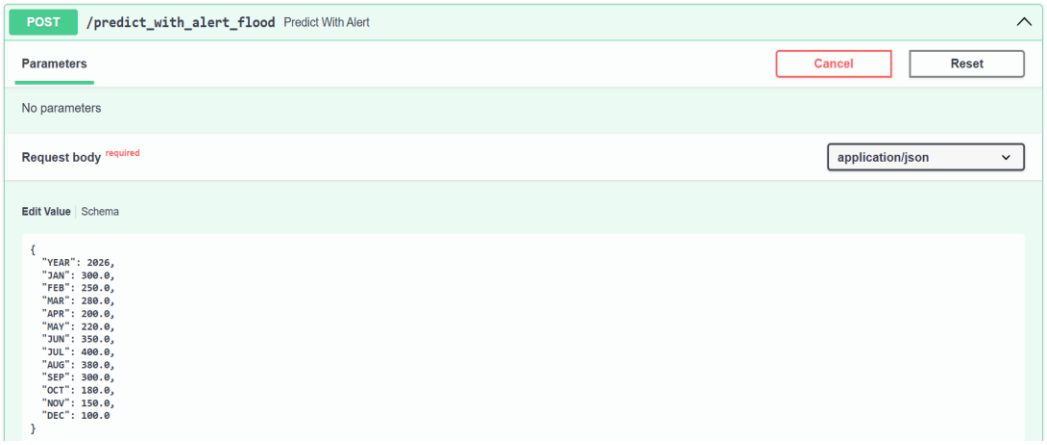
- AS:
  - if **risk_level = 0** → return "**Minor Earthquake Risk**"
  - if **risk_level = 1** → return "**Moderate Earthquake Risk**"
  - if **risk_level = 2** → return "**Severe Earthquake Warning**"



- Click **Execute**.
- You'll see the prediction in the **response body** below.



For FLOOD:



- Click **Execute**.
- You'll see the prediction in the **response body** below.

4.7.5. UI Agent Integration

The UI Agent represents the graphical user interface of the system.
It sends user input to the backend `/predict` API and displays the returned prediction, probability, and alert level to the user.
This ensures smooth interaction between the user and the deployed machine learning system.

4.7.6. Logging for Auditing

Each prediction request and its corresponding output (input data, prediction, probability, alert level, and timestamp) are logged into a database or file system.
These logs support auditing, performance analysis, and future retraining of the models.

4.7.7. Monitoring and Retraining

The deployed system is continuously monitored to detect data distribution shifts and performance degradation.
If model performance drops, retraining is performed using newly collected and labeled data.
Retraining is scheduled periodically, such as monthly or quarterly, depending on data availability.
(API request and response screenshots are included below to demonstrate successful deployment.)

FRONTEND

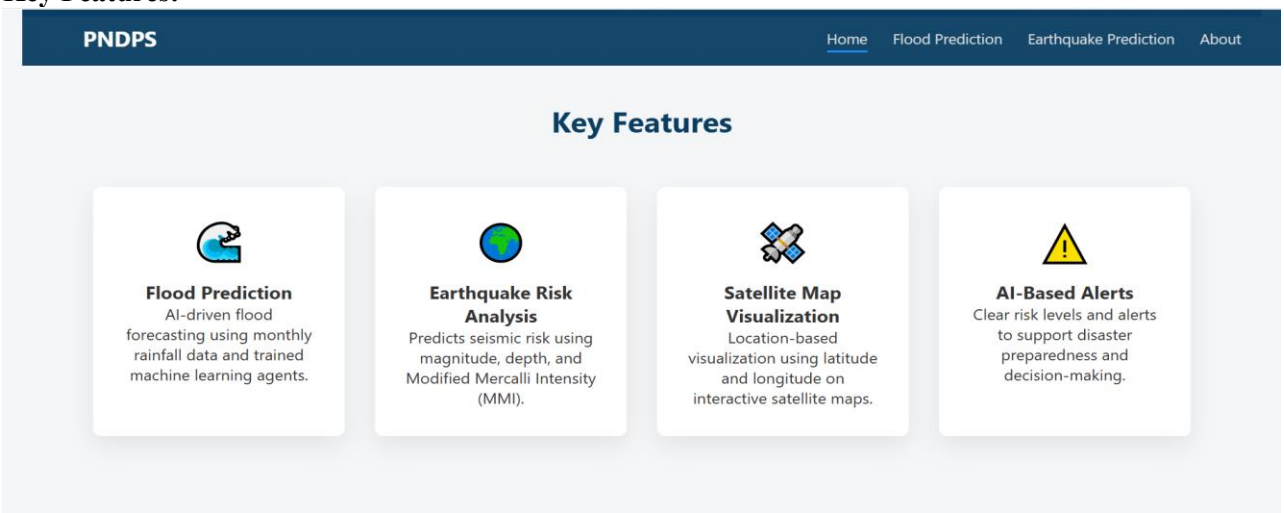
5. User Interface Design

5.1. Interface Overview

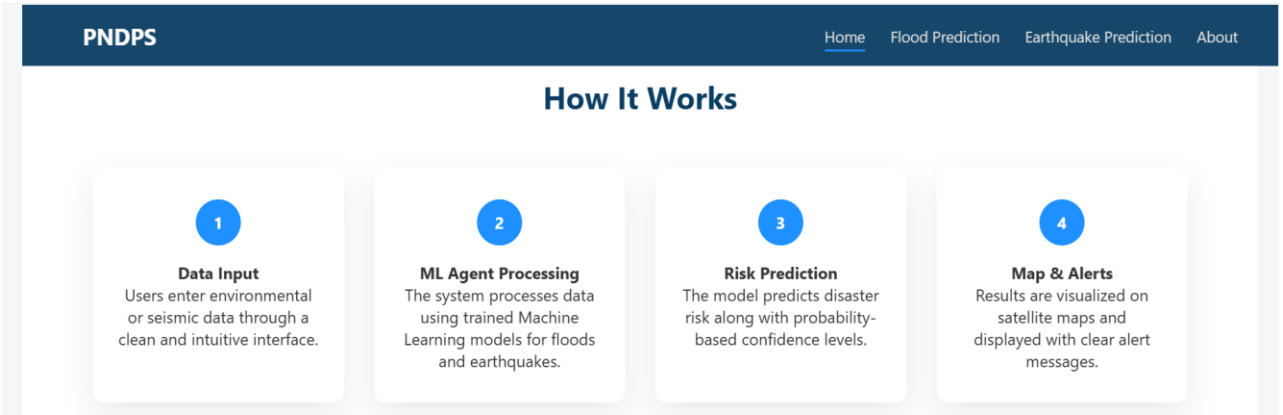
The Pakistan Natural Disaster Prediction System (PNDPS) features a **modern, professional, and responsive user interface** designed to provide users with easy access to flood and earthquake prediction tools.

Home Page:

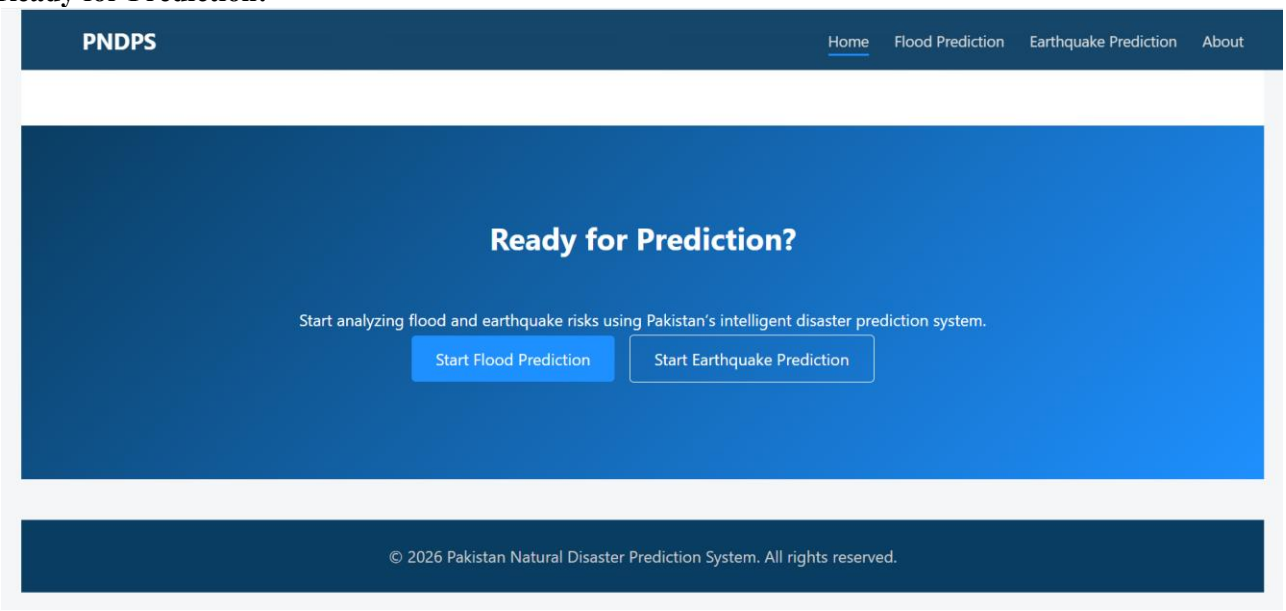
Key Features:



How it Works:

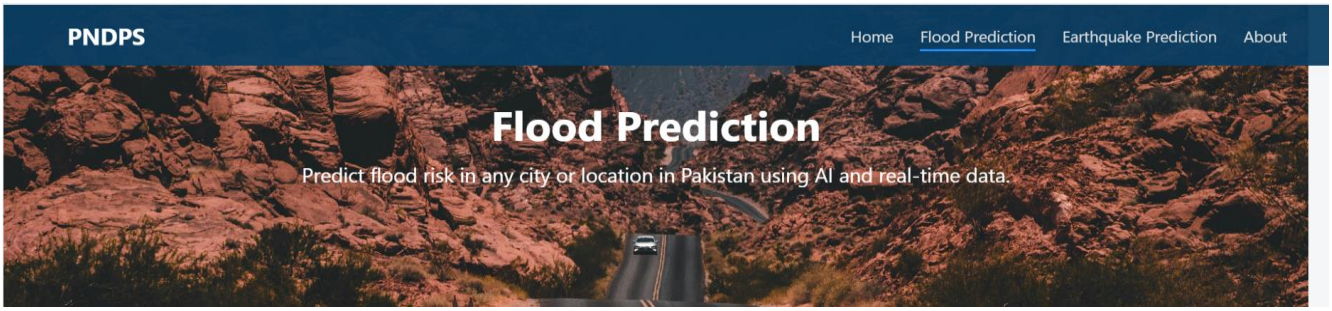


Ready for Prediction:



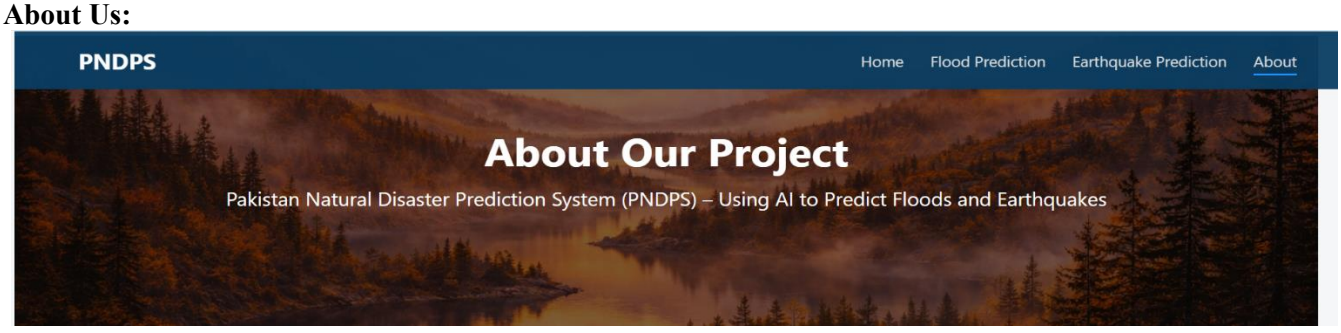
- The UI is divided into **three main modules**: Flood Prediction, Earthquake Prediction, and About Us.
- Each module features **distinctive backgrounds** (video or image), intuitive input sections, interactive maps, and visual results.
- The design is optimized for **desktop and mobile devices**, maintaining consistency in layout, color scheme, and typography.

Flood Prediction:



Earthquake Prediction:

About Us:



5.2. Input Options

The system provides a **structured input section** that allows users to enter relevant data for predictions.

Flood Prediction Input Fields:

- City Name (`<input type="text" id="city">`)
- Monthly Rainfall (`<input type="number" id="rainfall">`)
- Latitude (`<input type="number" id="latitude">`)
- Longitude (`<input type="number" id="longitude">`)

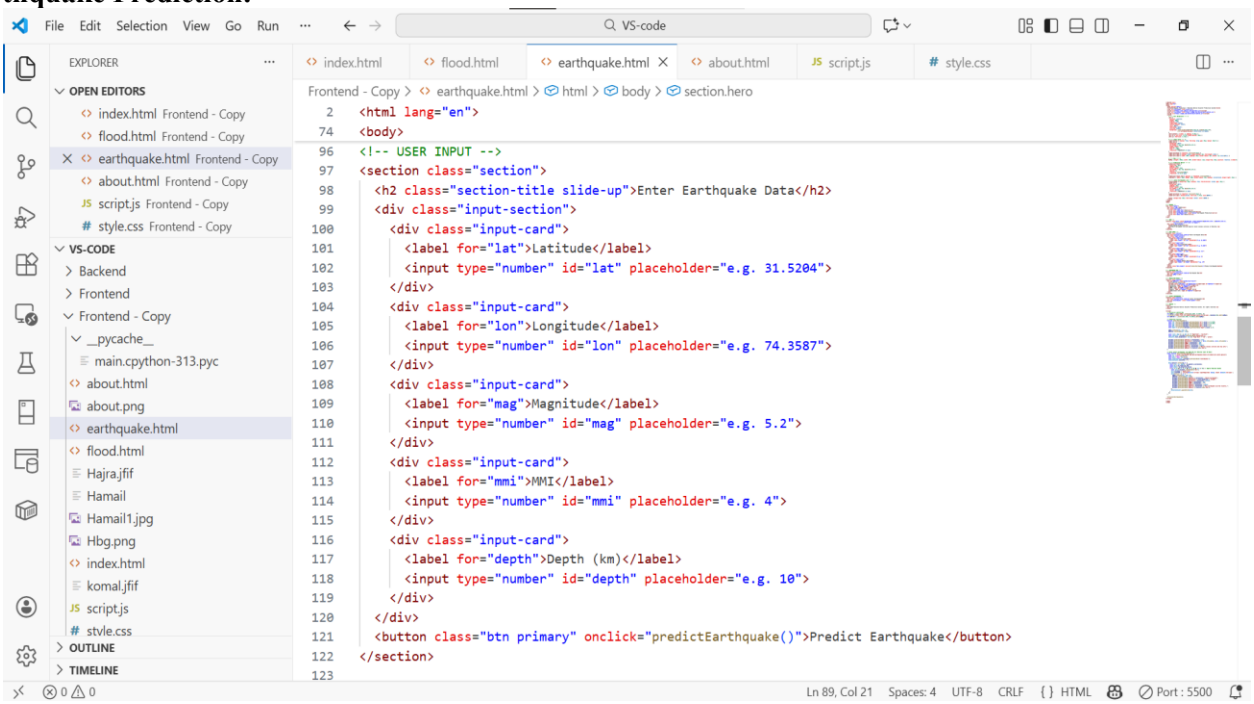
Earthquake Prediction Input Fields:

- Latitude (`<input type="number" id="lat">`)
- Longitude (`<input type="number" id="lon">`)
- Magnitude (`<input type="number" id="mag">`)
- MMI (`<input type="number" id="mmi">`)
- Depth in km (`<input type="number" id="depth">`)

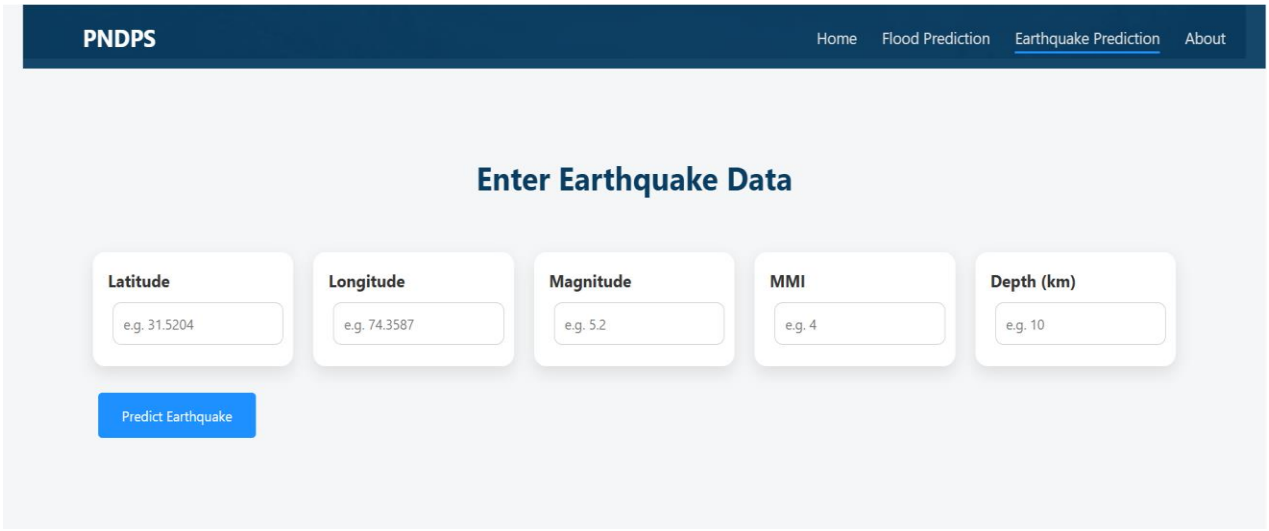
Design Notes:

- Each input is placed inside a **card with shadow, rounded corners, and hover effect**.
- Inputs are grouped in **flex layout** for clarity and responsiveness.
- Placeholder values guide the user for valid input format.

Earthquake Prediction:



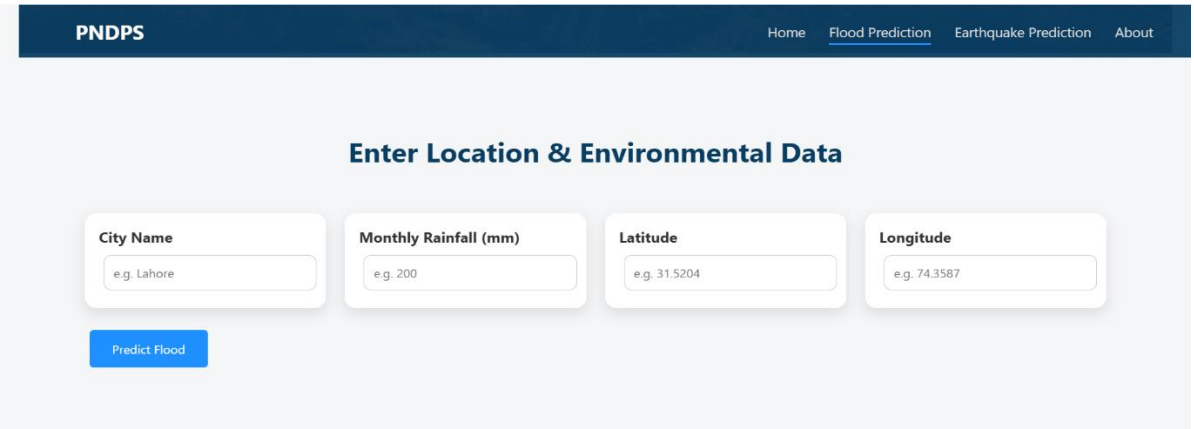
OUTPUT:



Flood Prediction:

```
<!-- USER INPUT -->
<section class="section">
  <h2 class="section-title slide-up">Enter Location & Environmental Data</h2>
  <div class="input-section">
    <div class="input-card">
      <label for="city">City Name</label>
      <input type="text" id="city" placeholder="e.g. Lahore">
    </div>
    <div class="input-card">
      <label for="rainfall">Monthly Rainfall (mm)</label>
      <input type="number" id="rainfall" placeholder="e.g. 200">
    </div>
    <div class="input-card">
      <label for="latitude">Latitude</label>
      <input type="number" id="latitude" placeholder="e.g. 31.5204">
    </div>
    <div class="input-card">
      <label for="longitude">Longitude</label>
      <input type="number" id="longitude" placeholder="e.g. 74.3587">
    </div>
  </div>
  <button class="btn primary" onclick="predictFlood()">Predict Flood</button>
</section>
```

OUTPUT:



5.3. Output Display

PNDPS provides **interactive and visual outputs** for predictions:

Flood Prediction Output:

- Prediction result card with **flood risk indicator** (red/green).
- Water level bar with animated fill.
- Distance to nearest river and additional information.
- Monthly rainfall chart (bar chart using Chart.js).
- Leaflet map shows the selected location with a marker.

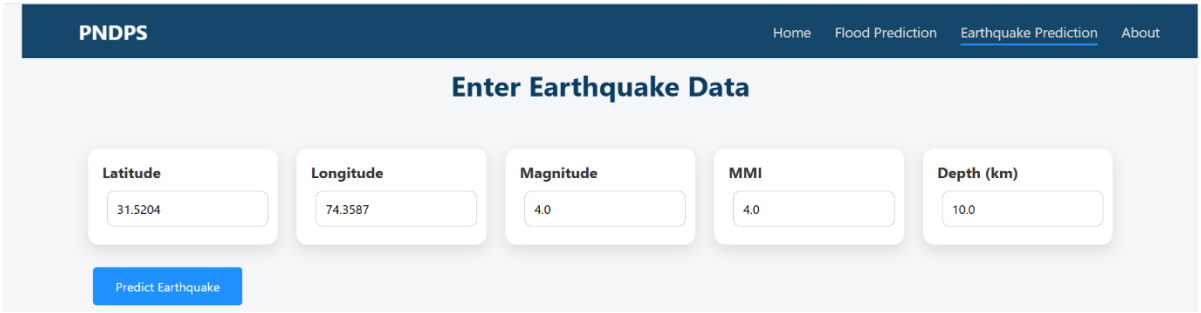
Earthquake Prediction Output:

- Prediction result card with **earthquake risk indicator**.
- Shows magnitude, MMI, depth, and location coordinates.
- Latest earthquakes list in Pakistan displayed dynamically; clicking an earthquake updates the map marker.
- Leaflet map for earthquake location.

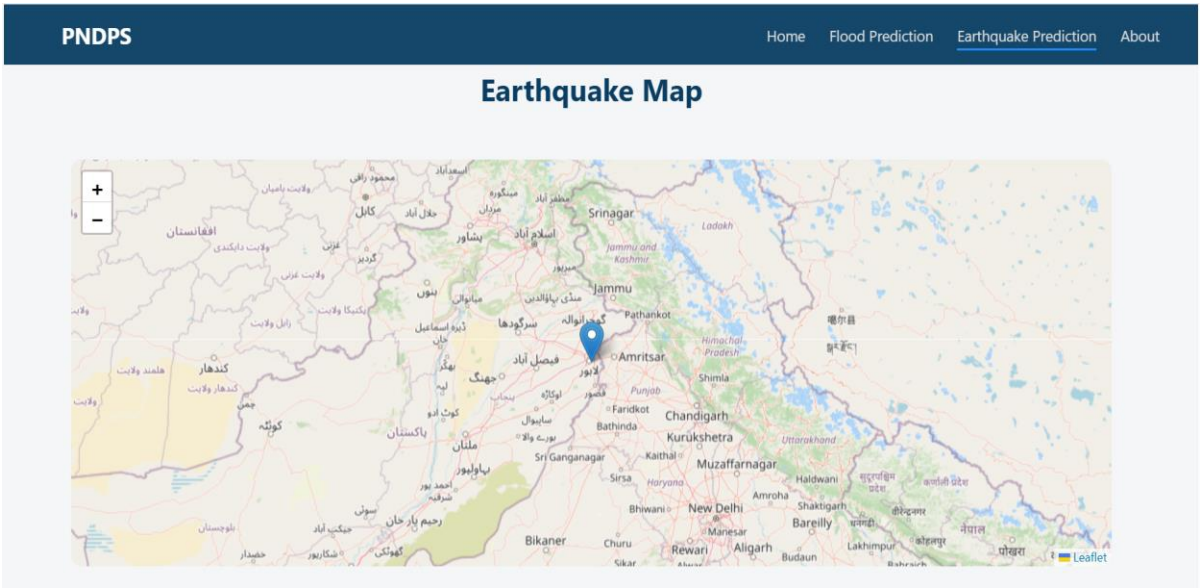
Earthquake Prediction:

```
<!-- EARTHQUAKE MAP -->
<section class="section">
  <h2 class="section-title slide-up">Earthquake Map</h2>
  <div id="eqMap"></div>
</section>

<!-- PREDICTION RESULT -->
<section class="section">
  <div id="eqResult" class="prediction-result">
    <h3>Prediction Result</h3>
    <p><span class="indicator" id="eqIndicator"></span><span id="eqStatus"></span></p>
    <p>Location: <span id="eqLocation"></span></p>
    <p>Magnitude: <span id="eqMag"></span></p>
    <p>MMI: <span id="eqMMI"></span></p>
    <p>Depth: <span id="eqDepth"></span> km</p>
    <p>Additional Info: <span id="eqExtra"></span></p>
  </div>
</section>
```

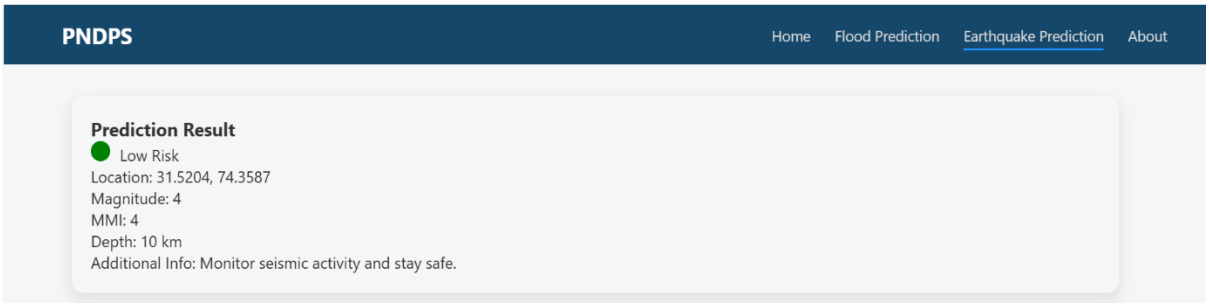
OUTPUT:



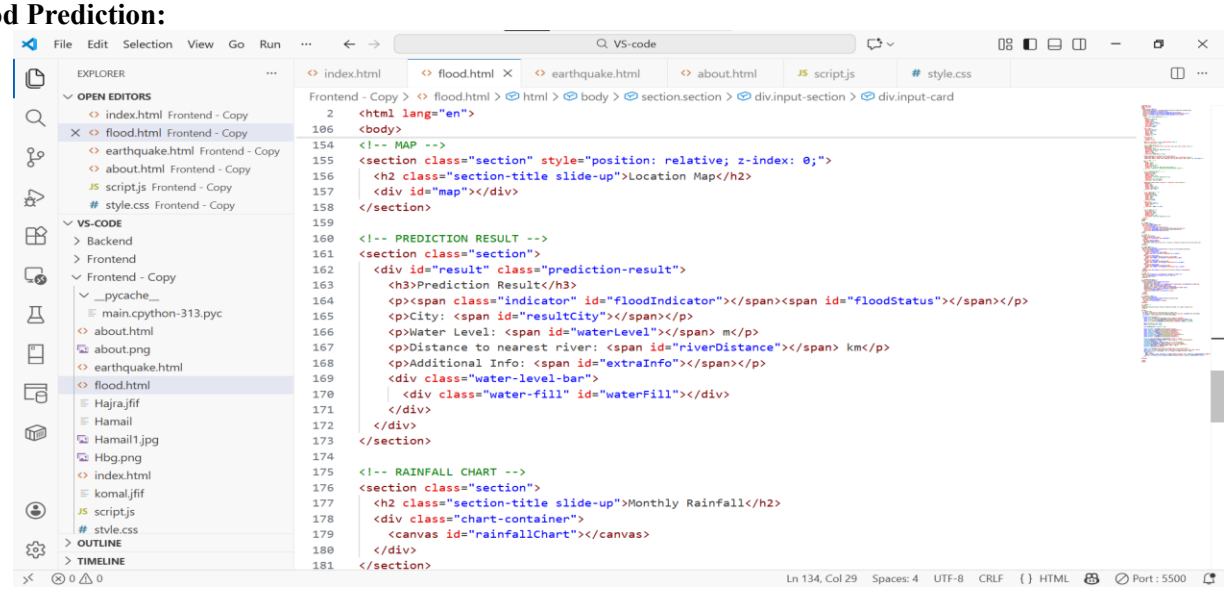
MAP:



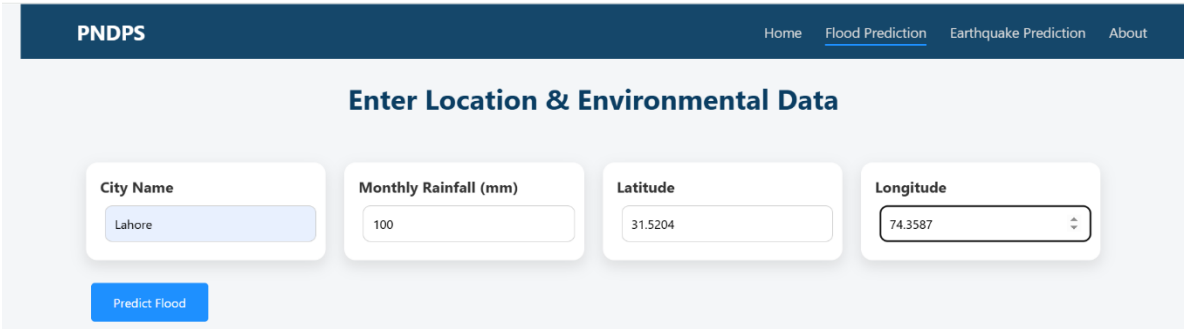
Prediction:



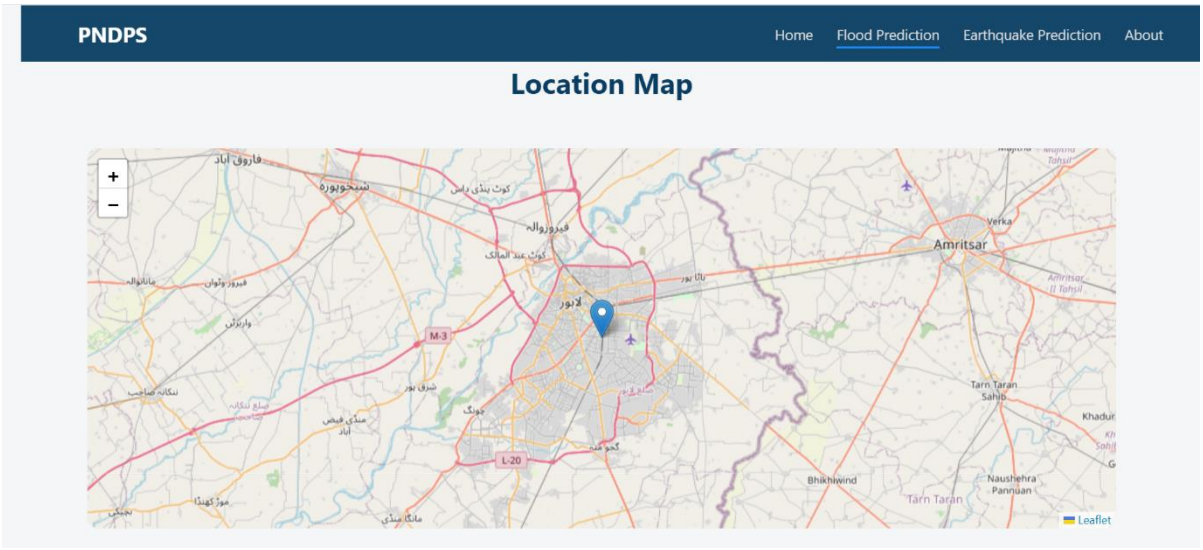
Flood Prediction:



OUTPUT:

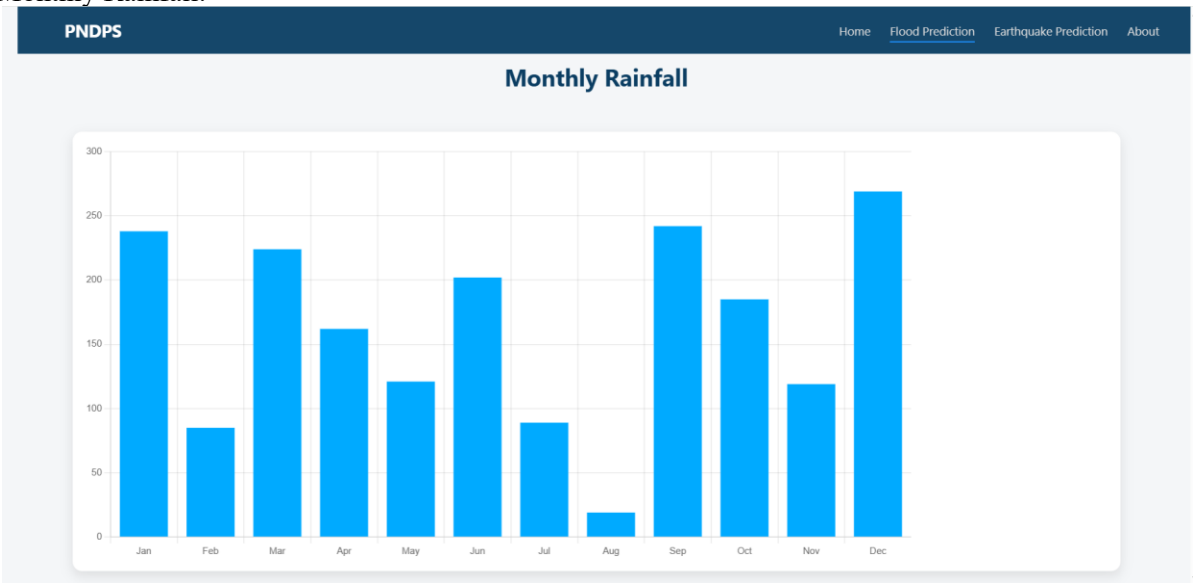


MAP:



Prediction:

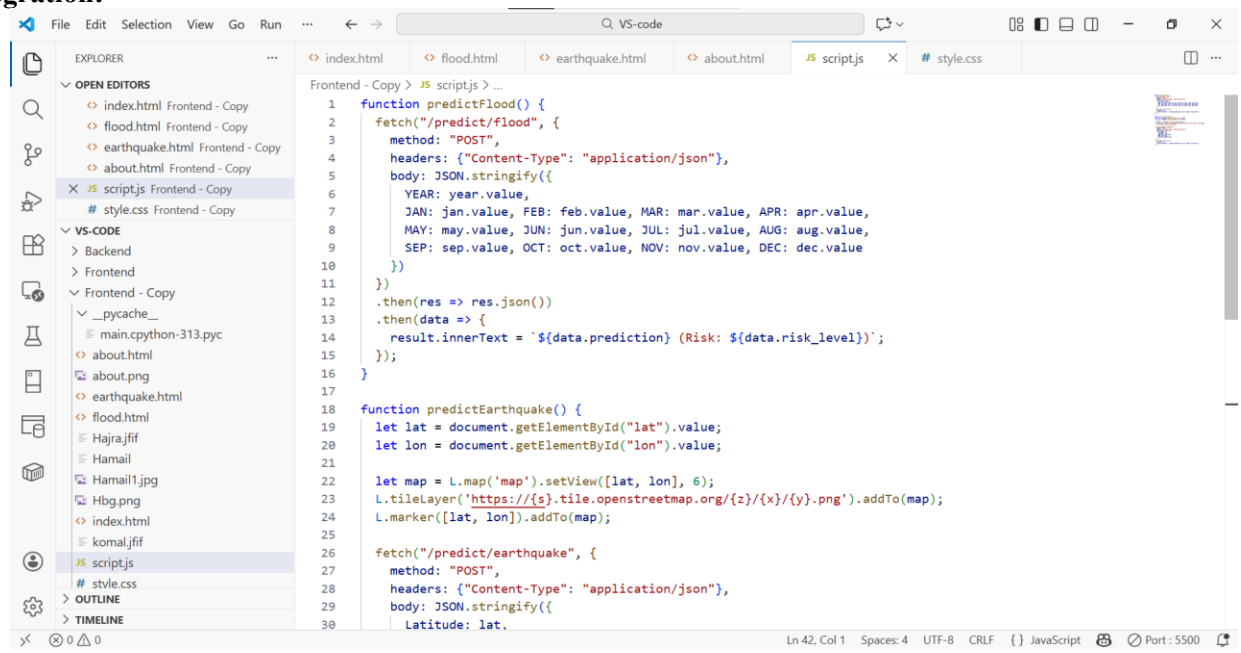
Monthly Rainfall:



5.4. Frontend & Backend Integration

- **Frontend:** HTML, CSS, and JavaScript handle **UI rendering, animations, and user input validation**.
- **Backend:** Python (Flask or FastAPI) handles **ML model predictions** and sends results via API endpoints.
- The system fetches **live data from APIs**:
  - Rainfall and river data for flood predictions.
  - Latest earthquake information for seismic activity monitoring.
- **Integration Flow:**
  1. User enters input → frontend sends API request → backend processes data → returns prediction → frontend updates UI dynamically.
- **Interactive Maps:** Leaflet.js maps update markers based on user input or selected latest earthquake.

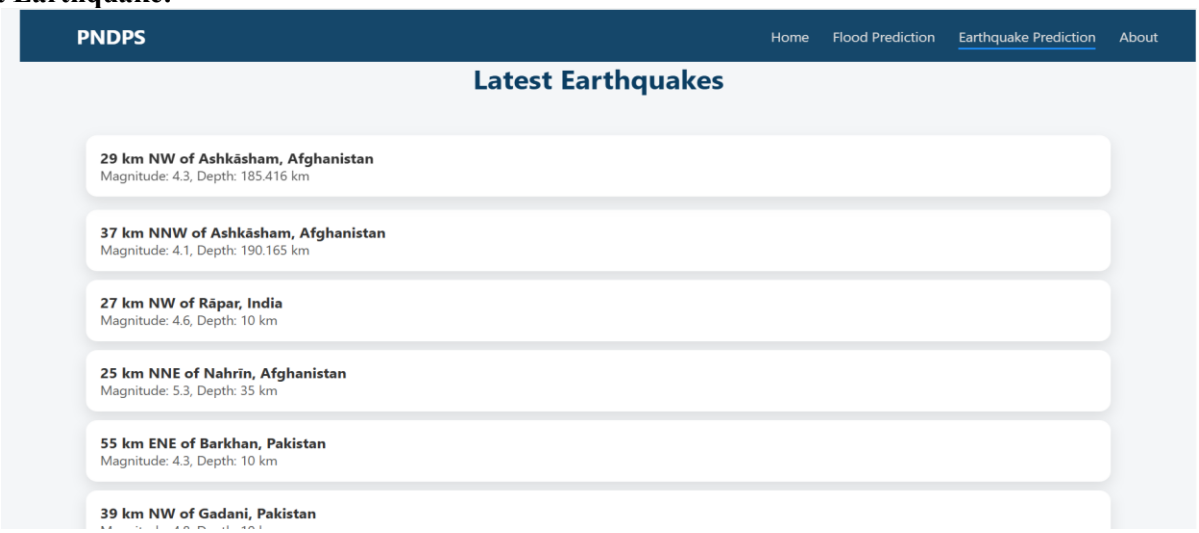
Integration:



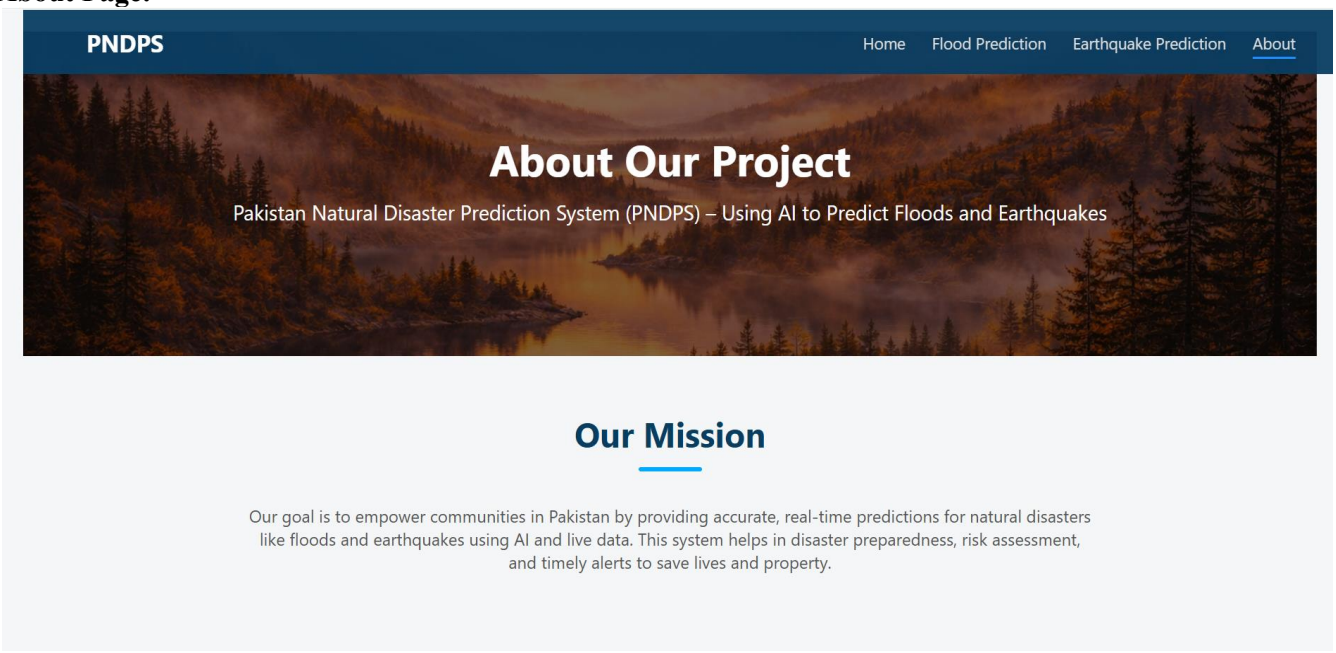
5.5. User Experience Design

- The design emphasizes **clarity, responsiveness, and accessibility**.
- **Animated transitions** for hero sections, input cards, and result cards improve engagement.
- Backgrounds (video for flood, image for earthquake, and gradient overlays) enhance visual appeal without distracting from content.
- Team section and contact section are structured for **quick reference** with hover effects and social media links.
- Users receive **immediate feedback** via colored indicators, charts, and dynamic maps.
- **Responsive design** ensures usability across devices: mobile, tablet, and desktop.

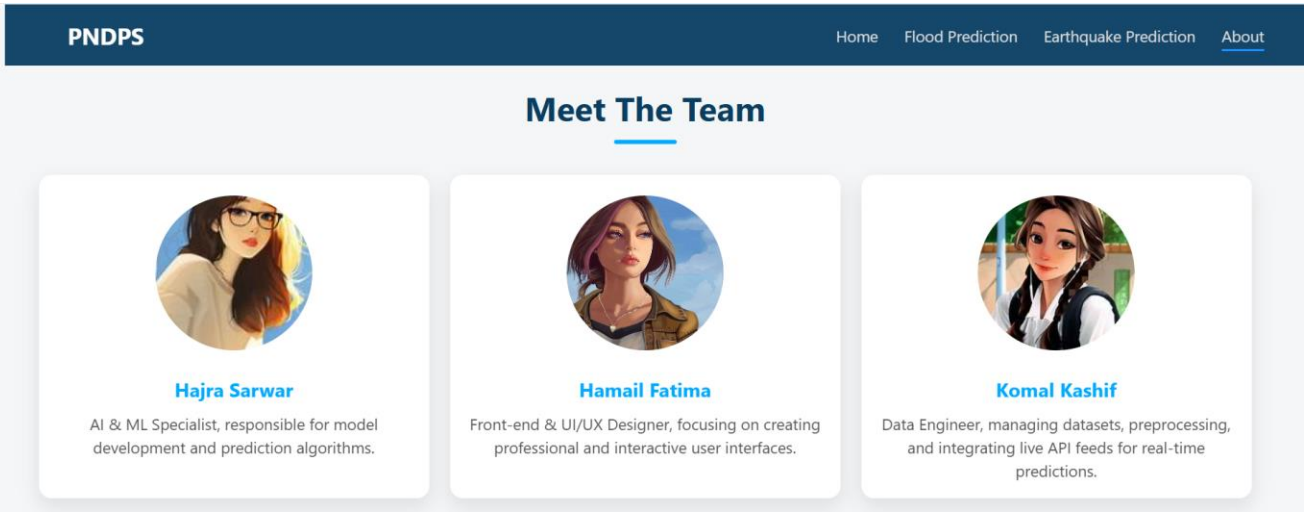
Latest Earthquake:



About Page:



Team:



Contact Us:

6. Code and Implementation

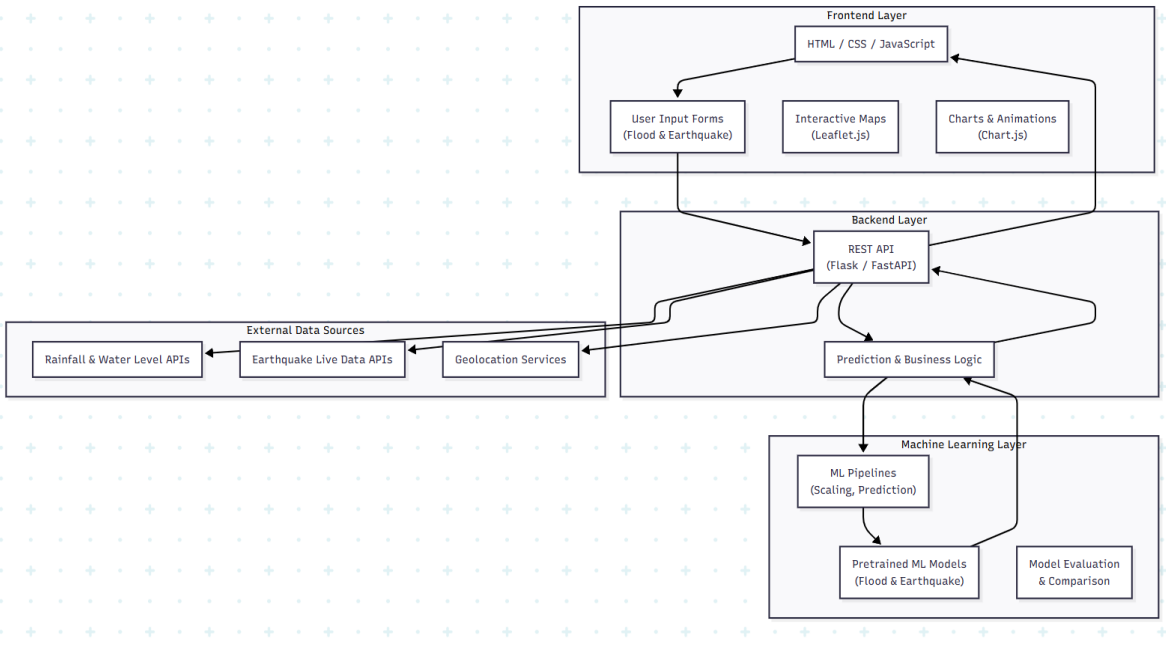
6.1. System Architecture Diagram and Modules

The Pakistan Natural Disaster Prediction System (PNDPS) follows a **modular, layered architecture** that separates concerns between data processing, machine learning, backend services, and frontend visualization.

Architecture Layers:

1. **Frontend Layer**
   - HTML, CSS, JavaScript
   - User input forms (Flood & Earthquake)
   - Interactive maps (Leaflet.js)
   - Charts and animations (Chart.js)
2. **Backend Layer**
   - Python-based REST API (Flask / FastAPI)
   - Handles user requests and prediction logic
   - Integrates ML models and live data APIs
3. **Machine Learning Layer**
   - Pretrained ML models (Flood & Earthquake)
   - Pipelines for preprocessing and prediction
   - Model comparison and evaluation logic
4. **External Data Sources**
   - Rainfall and water level APIs
   - Earthquake live data APIs
   - Geolocation services

Diagram:



6.2. File / Folder Structure and Key Scripts

The project is organized using a **clean and scalable folder structure** to improve maintainability and readability.

Folder Structure:

```
PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project\VS-code\Frontend> cd "D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project"
PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project> ls


    Directory: D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project


d-----        12/6/2025  11:51 AM              Articles
d-----         1/2/2026   8:01 PM              DataSet
d-----         1/4/2026  12:06 PM              Document
d-----        12/6/2025  11:53 AM              Senior-Word
d-----         1/4/2026  12:56 AM              VS-code



    Directory: D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project\VS-code


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         1/4/2026   7:35 AM                Backend
d-----         1/4/2026   1:02 AM                Frontend
d-----         1/4/2026  10:52 AM                Frontend - Copy
-a----         1/2/2026   1:34 PM           5319 np.html
-a----         1/2/2026   1:24 PM         445648 WhatsApp Image 2026-01-02 at 1.00.26 PM.jpeg
-a----         1/2/2026   1:24 PM          94848 WhatsApp Image 2026-01-02 at 1.07.39 PM.jpeg
-a----         1/2/2026   1:24 PM         144531 WhatsApp Image 2026-01-02 at 1.07.41 PM.jpeg
-a----         1/2/2026  12:45 PM        1481729 WhatsApp Video 2026-01-02 at 12.44.16 PM.mp4
```

```
PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project\VS-code> cd "Backend"
PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project\VS-code\Backend> ls


    Directory: D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project\VS-code\Backend


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        12/24/2025   2:01 AM                .venv
d-----        12/24/2025   2:07 AM                .vscode
d-----         1/2/2026   4:21 PM                api
d-----        12/25/2025  11:25 PM                data
d-----         1/3/2026  10:53 AM                models
d-----         1/2/2026   8:52 PM                notebooks
d-----        12/27/2025   1:11 AM                pipelines
d-----         1/4/2026   7:35 AM                __pycache__
-a----         1/3/2026  12:30 AM            670 install_deps.py
-a----         1/3/2026  12:08 AM            553 install_tf.py
-a----         1/4/2026   7:32 AM           1732 main.py
-a----        12/15/2025   6:38 PM              0 README.md
-a----        12/28/2025   1:06 PM             96 requirements.txt
-a----        12/27/2025   2:47 AM            796 test_pipeline.py
```

```
PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project> cd "VS-code\Frontend - Copy"
PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project\VS-code\Frontend - Copy> ls


    Directory: D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project\VS-code\Frontend - Copy


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         1/4/2026   7:32 AM                __pycache__
-a----         1/4/2026  10:54 AM           5729 about.html
-a----         1/4/2026  10:50 AM        2698929 about.png
-a----         1/4/2026  10:54 AM           8523 earthquake.html
-a----         1/4/2026   9:57 AM           7834 flood.html
-a----         1/4/2026  10:20 AM           5176 Hajra.jfif
-a----         1/4/2026  10:19 AM          89896 Hamail
-a----         1/4/2026  10:23 AM          89896 Hamail1.jpg
-a----         1/4/2026   6:26 AM        2693721 Hbg.png
-a----         1/4/2026  11:23 AM           4404 index.html
-a----         1/4/2026  10:19 AM          10247 komal.jfif
-a----         1/4/2026   7:39 AM           1250 script.js
-a----         1/4/2026   8:14 AM           6334 style.css
-a----         1/4/2026   2:08 AM       17874845 water.mp4


PS D:\FJWU\Semester 5\Artificial Intelligence-Dr. Irum Matloob\Project\VS-code\Frontend - Copy>
```

Key Scripts:

- `app.py`: Main backend server entry point
- `flood_pipeline.pkl`: Saved ML pipeline for flood prediction
- `disaster_agent.py`: Automation and decision logic
- `train_flood_model.py`: Flood model training script

6.3. Backend Code Summary (Services and Endpoints)

The backend acts as a **bridge between frontend UI and ML models**.

Backend Responsibilities:

- Receive user input via REST APIs
- Validate and preprocess data
- Load trained ML pipelines
- Generate predictions
- Send formatted responses to frontend

Technologies Used:

- Python
- Flask / FastAPI
- Joblib / Pickle for model loading
- JSON for data exchange

Example Backend Flow:

1. User submits form
2. API endpoint receives request
3. ML pipeline predicts outcome
4. Response returned to frontend

6.4. ML Model Code Structure (Pipelines, Training Scripts)

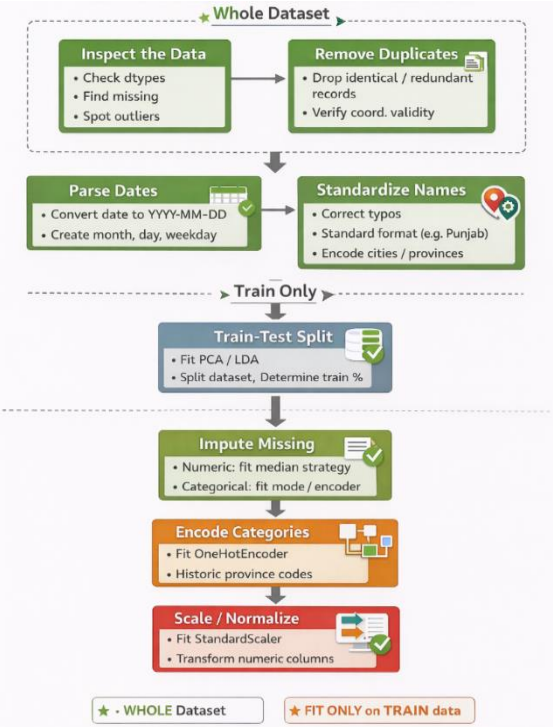
The ML layer is designed using **pipelines for consistency and reproducibility**.

Flood Prediction Models:

- Logistic Regression
- Random Forest
- Decision Tree
- KNN

ML Pipeline Structure:

- Data Cleaning
- Feature Selection
- Scaling (MinMaxScaler)
- Model Training
- Model Evaluation
- Pipeline Saving



6.5. Agent Reasoning Code (Automation Logic)

An intelligent **Disaster Prediction Agent** automates system behavior based on model outputs.

Agent Responsibilities:

- Interpret prediction results
- Assign risk levels
- Trigger alerts (Flood / Earthquake)
- Update UI indicators dynamically

Example Logic:

- If flood probability > threshold → "Flood Expected"
- If magnitude ≥ 5 or MMI ≥ 5 → "High Earthquake Risk"

Agent Advantages:

- Reduces manual intervention
- Ensures consistent decision-making
- Enables future automation (alerts, notifications)

6.6. API Endpoints and Sample Requests / Responses

The system exposes REST APIs for frontend communication.

Flood Prediction API:

Endpoint:

```
POST /predict_with_alert_flood
```

Sample Request:

```
{
  "city": "Lahore",
```

```
  "rainfall": 220,
  "latitude": 31.52,
  "longitude": 74.35
}
```

Sample Response:

```
{
  "alert": "Flood Expected",
  "risk_level": 1,
  "class_probabilities": [0.2, 0.8]
}
```

Earthquake Prediction API:

Endpoint:

`POST /predict_earthquake`

Sample Request:

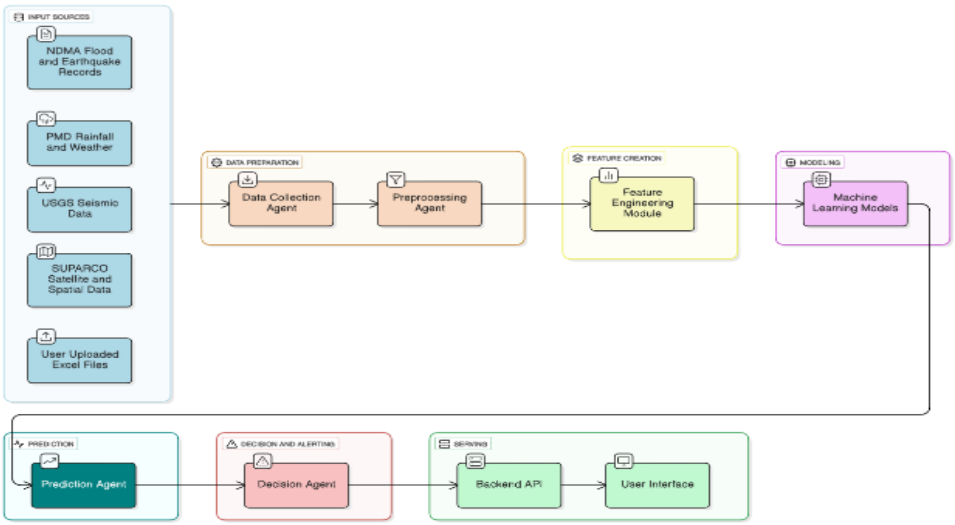
```
{
  "latitude": 33.7,
  "longitude": 73.1,
  "magnitude": 5.4,
  "mmi": 5,
  "depth": 12
}
```

Sample Response:

```
{
  "risk": "High Risk",
  "severity": "Moderate to Strong",
  "recommended_action": "Stay Alert"
}
```

Live Earthquake Data API:

- Fetches latest earthquakes in Pakistan
- Updates map on user click
- Displays magnitude, depth, MMI, and location



---

7. Deployment & Monitoring (Operations)

This section describes how the Pakistan Natural Disaster Prediction System (PNDPS) is deployed, monitored, maintained, and managed in a production-like environment to ensure reliability, scalability, and continuous improvement.

7.1. Deployment Steps and Runbook

The deployment of PNDPS follows a structured and repeatable process to ensure consistency across environments.

Deployment Environment

- **Frontend**: Hosted as a static web application using HTML, CSS, and JavaScript.
- **Backend**: Python-based REST API (Flask/FastAPI).
- **Machine Learning Models**: Pretrained models stored as serialized pipelines.
- **External APIs**: Integrated for live rainfall, water-level, and earthquake data.

Deployment Steps

1. **Environment Setup**
    - Install Python dependencies (ML libraries, API frameworks).
    - Configure environment variables for API keys and endpoints.
2. **Model Deployment**
    - Load pretrained flood and earthquake models.
    - Validate model integrity and compatibility.
3. **Backend Deployment**
    - Deploy REST API services.
    - Verify prediction endpoints and live data connections.
4. **Frontend Deployment**
    - Upload UI files.
    - Ensure correct API endpoint bindings.
5. **Integration Testing**
    - Verify real-time data fetching.
    - Validate prediction flow end-to-end.
6. **Production Release**
    - Enable public access.
    - Monitor system health post-release.

Runbook

- Restart backend services on failure.
- Check API connectivity if predictions fail.
- Reload ML models if version mismatch occurs.

7.2. Monitoring Dashboards and Alerts

To maintain operational stability, PNDPS uses continuous monitoring and alerting mechanisms.

Monitoring Metrics

- API response time
- Prediction success rate
- Live data fetch latency
- Error and exception frequency
- System uptime

Dashboards

- **Backend Health Dashboard**: Displays API status and request volumes.
- **Model Performance Dashboard**: Tracks prediction accuracy trends.
- **Live Data Dashboard**: Shows API availability and freshness.

Alerts

- Triggered when:
    - API response time exceeds threshold
    - External data source becomes unavailable
    - Prediction service crashes
- Alerts are logged and sent to system administrators for rapid action.

7.3. Retraining Policy and Lifecycle Management

PNDPS follows a controlled machine learning lifecycle to maintain model accuracy.

Retraining Policy

- Models are retrained:
  - Periodically (e.g., quarterly)
  - After significant data drift
  - Following major natural disaster events
- New data is validated before training.

Lifecycle Management

1. Data Collection
2. Data Preprocessing
3. Model Training and Evaluation
4. Model Validation
5. Deployment of Updated Model
6. Archival of Old Model Versions

Version Control

- Each model version is documented.
- Rollback is possible if performance degrades.

7.4. Incident Response and Rollback Plan

PNDPS includes a structured incident response strategy to minimize downtime.

Incident Types

- API downtime
- Incorrect predictions
- External data source failure
- Model performance degradation

Response Procedure

1. Detect issue via monitoring alerts.
2. Isolate affected component.
3. Switch to fallback logic (cached data or last stable model).
4. Notify administrators.
5. Apply fix and test.
6. Restore normal operation.

Rollback Strategy

- Revert to last stable model version.
- Restore previous backend configuration.
- Disable faulty external API temporarily if needed.

---

8. Conclusion & Future Work

8.1. Summary of Results and Contributions

The Pakistan Natural Disaster Prediction System successfully demonstrates the integration of artificial intelligence with real-time environmental data to predict floods and earthquakes.

Key contributions include:

- AI-based flood and earthquake prediction models.
- Real-time data integration for enhanced situational awareness.
- Interactive visualizations using maps and charts.
- Modular, scalable system architecture.

The system provides a strong foundation for intelligent disaster preparedness in Pakistan.

8.2. Limitations and Ethical Considerations

Limitations

- Prediction accuracy depends on data quality.
- External APIs may experience downtime.
- Models may not capture rare or extreme events accurately.
- Limited historical data for certain regions.

Ethical Considerations

- Predictions are advisory, not absolute.
- Users must be informed about uncertainty.
- No personal data is collected or stored.
- System avoids inducing panic through responsible messaging.

8.3. Recommended Future Improvements

Future enhancements may include:

- Integration with official government warning systems.
- Deep learning models for improved accuracy.
- Mobile application deployment.
- SMS and push notification alerts.
- Multilingual support (Urdu and regional languages).
- Advanced risk visualization dashboards.
- Automated model retraining pipelines.

---

9. References

1. World Meteorological Organization (WMO) – Flood Forecasting Guidelines
2. United States Geological Survey (USGS) – Earthquake Data and Models
3. OpenWeatherMap API Documentation
4. Scikit-learn: Machine Learning in Python
5. Leaflet.js Documentation
6. Chart.js Documentation
7. IEEE Standards for Software Documentation
